ER and Relational ModelUnit-2)

What is Data Modeling?

Data modeling is a process that helps you to understand the structures, form, and relationships of information. Just like any model, it's a simplified and imperfect version of reality. But with careful and considerate work, a well-designed data model will be able to respond to change and handle future requirements.

Conceptual vs Logical vs Physical Data Modeling

There are three traditional levels of data modeling. Not every team will necessarily follow all three strictly. Often, all three – conceptual, logical, and physical data models – are compressed into one modeling exercise.

However, breaking the process down into these three levels can be valuable. Each step lays down a foundation for the next:

- Conceptual – the "what" model
- Logical – the "how" of the details
- Physical – the "how" of the implementation

Each level of conceptual, physical, and logical data models can involve different roles from your team.

Conceptual Data Model

The conceptual data model can be thought of as the "white board" data model. This model does not go into the "how" at all.

For this model, it's important to focus on capturing all the types of data (or "entities") that the system will need. In addition to entities, a conceptual data model will also capture:

- **Attributes:** individual properties of an entity. For instance, a "person" entity may have "name" and "shoe size". An "address" entity may have "zip code" and "city".

- **Relationships:** how an entity connects to other entities. For instance, a "person" entity may have one or more "addresses".Along with the entities, their attributes and relationships, a conceptual model can also:

- **Organize scope:** which entities are included, but also which are explicitly NOT included.

- **Define business concepts / rules:** For instance, are person entities allowed to have multiple addresses? What about multiple emails? Do they need to have a unique identifier?

The conceptual data model is often created by architects in conjunction with business stakeholders and domain experts.
Conceptual Data Model Example

Definitions		
Entity	**Description**	**Relationships**
Hotel	Physical location of the hotel with is key in identify a hotel. Hotels have many guest rooms. Reservations is for guest not conference rooms (not shown here).	Must have many rooms May have many customers
Room	Rooms are a sub-component of a hotel. They cannot exist outside of the hotel. Rooms are reserved by a guest.	Reserved by many guests
Guest	A guest reserves are room at the hotel. They can also be a customer. The difference between a guest and customer is that customers don't necessarily have to reserve a room to transact with the hotel.	Reserves a room Can be a customer
Customer	A customer transact business with the hotel but is not required to be a guest. Customers could purchase from the gift shop.	Transacts with the hotel
Reservation	A reservation reserves a room from the inventory of available rooms for a guest. It is uniquely identified by a confirmation number. Guest can make multiple reservations	Reserves a room for a guest.

There are many “languages” for describing a conceptual data model. But as long as it’s documented in an accessible way, it can be as easy as boxes and arrows.

Here's an example of a conceptual diagram that involves two core entities, *travel routes* (and its associated schedules) and *airlines*:

While these may look like tables in a relational database, the conceptual modeling stage is too early to make a determination about *how* the data will be stored. That determination comes later: it could be tables, JSON documents, graph nodes, CSV files, blockchain, or any other number of storage mediums.

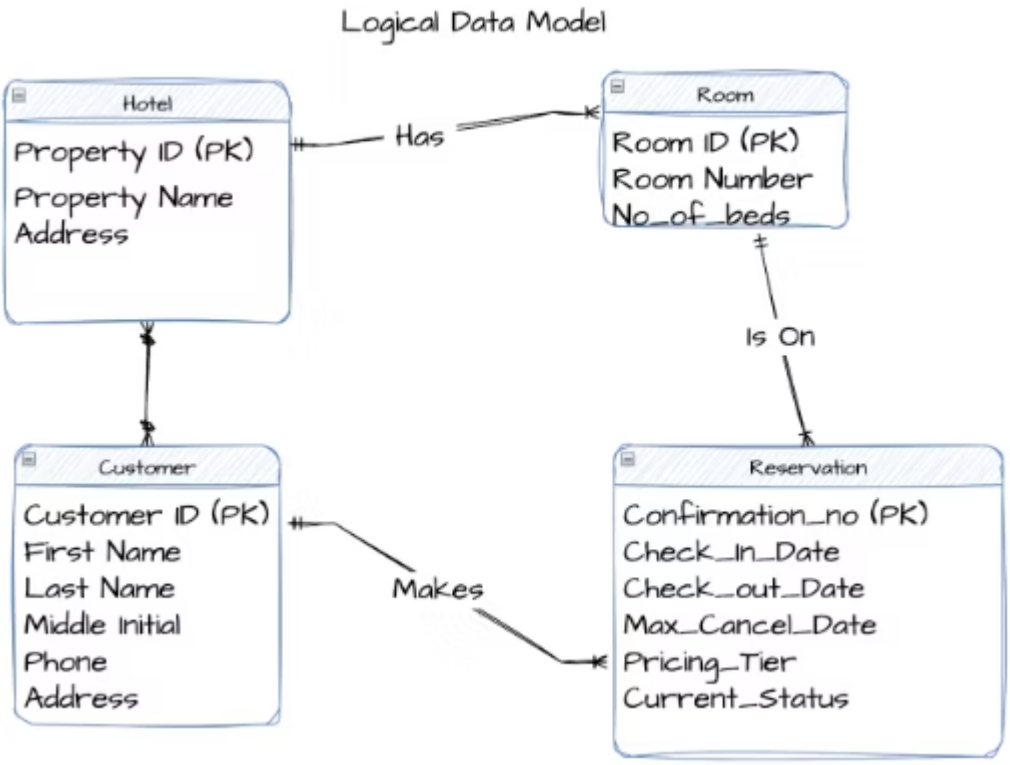
Logical Data Model

A logical data model is the next step, once the stakeholders have agreed on a conceptual model. This step involves filling in the *details* of the conceptual model. It's still too early to pick a specific DBMS, but this step can help you decide which *class* of database to use (relational, document, etc). For instance, if you decide **relational**, then it's time to decide which tables to create. If you decide **document**, then it's time to define the collections.

Decide the details of each individual field/column and relationship as well. This includes data types, sizes, lengths, arrays, nested objects, etc.

The logical model is typically created by architects and business analysts.

Logical Data Model Example



For instance, if going with a relational model, the logical model might look like this:

However, with a document database, the schedule can be modeled as part of the route, directly. No need for a foreign key, but it's still helpful to think of it as its own sub-entity. So that logical model might look like this:

In the case of a schedule, which has a fairly small, finite footprint, embedding into the same collection makes sense. In the case of, for example, a user making social media posts, which is unbounded, it makes sense to model in separate collections.

Physical Data Model

Once a logical model has been defined, it's now time to actually implement it into a real database.

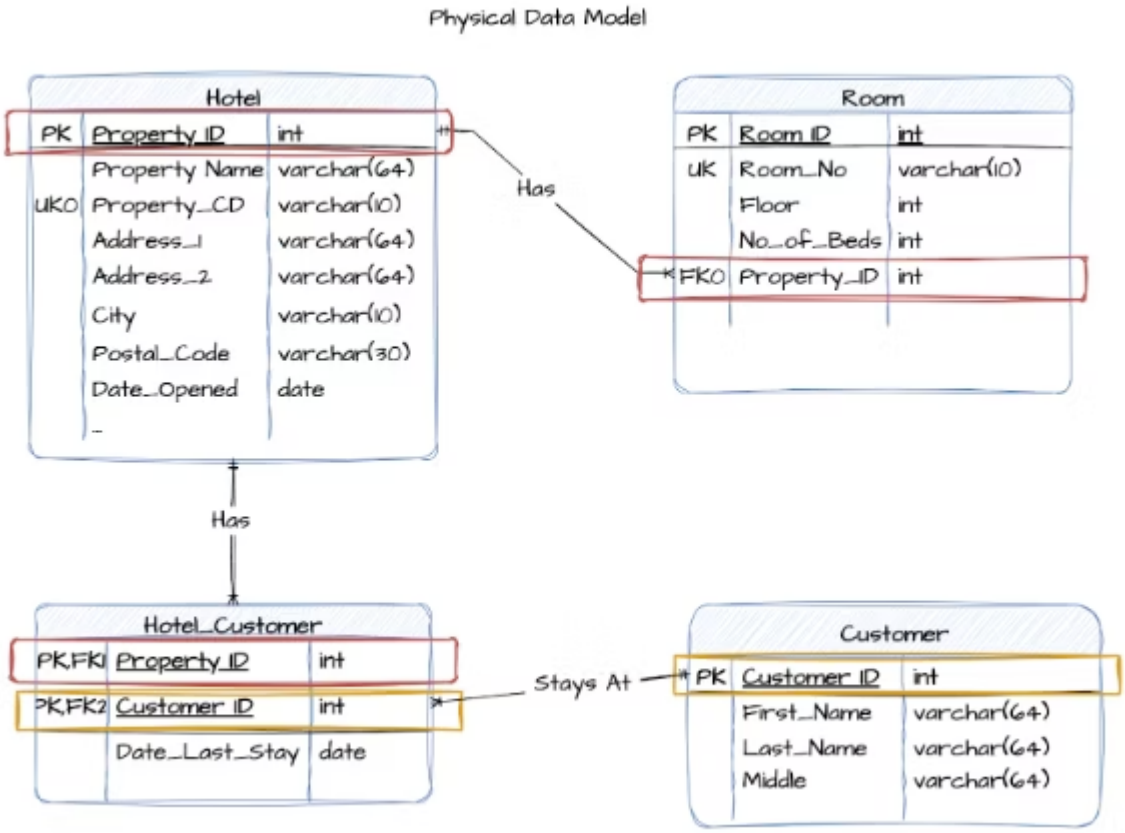
If you decided on a relational model, options include SQL Server, Oracle, PostgreSQL, MySQL, etc. However, if your modeling process reveals that your data model is likely to change frequently to adapt to new requirements, you might still consider going with a document database. One of the best choices for this is Couchbase, a "NoSQL" document database that supports familiar relational concepts like JOINs, ACID transactions **and** flexible JSON data.

The physical data model should include:

- A specific DBMS (Couchbase, for instance)
- How data is stored (On disk/RAM/hybrid/etc. Couchbase has a built-in cache to provide the speed of RAM with the durability of disk)
- How to accommodate replications, shards, partitions, etc. (For Couchbase, sharding and partitioning is automatic. Replication is a drop-down box to select how many replicas you want).

The physical data model is typically created by DBAs and/or developers.

Physical Data Model Example



Here's an example of a physical model for Couchbase:

It's sometimes helpful to show sample data along with the physical model.

What is ER Modeling?

A graphical technique for understanding and organizing the data independent of the actual database implementation. We need the Entity-Relationship (ER) model to help us design and organize data in a way that is easy to understand and maintain.

The ER model provides a graphical representation of the data and the relationships between entities, attributes, and constraints in a database.

ER modeling is the process of creating an ER model by identifying the entities, attributes, and relationships that exist within a system or organization.

In ER modeling, entities represent objects or concepts in the real world, such as customers, orders, or products. Each entity is described by a set of attributes, which represent the characteristics of the entity.

For example, a customer entity might have attributes such as name, address, and phone number.

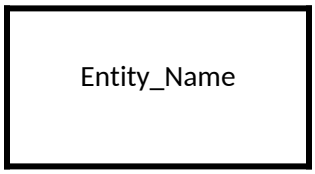
Relationships describe how entities are related to each other. A relationship between two entities represents a link or association between them. Relationships are classified based on the number of entities involved and the cardinality of the relationship (e.g., one-to-one, one-to-many, many-to-many).

Entity sets

An entity set is a collection of similar entities. An entity set is a set of entities of the same type that share the same properties,or attributes.

For example, in a university database, the entity set "students" represents all the individual students who attend the university.

Entity sets are typically represented as rectangles in an ER diagram, with the entity set name written inside the rectangle.



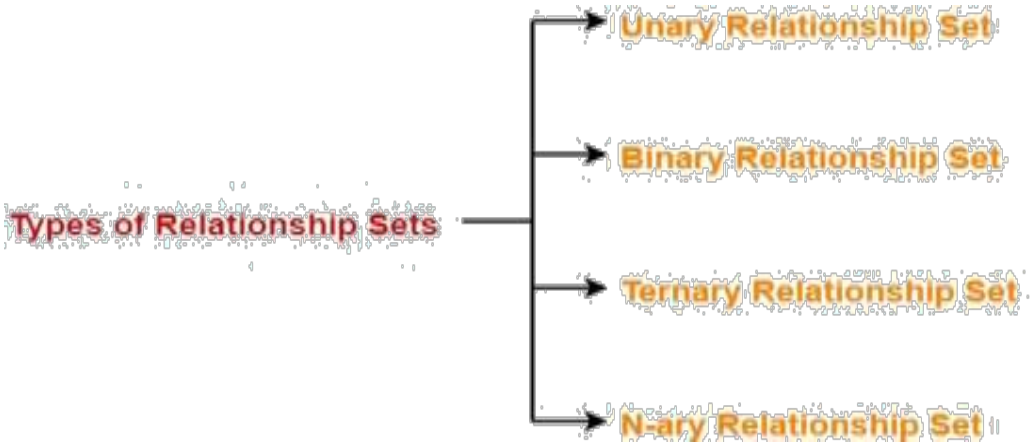
Types of Entities: -

- **Strong entity:** A strong entity is an entity that exists independently of other entities. It has its own unique identifier and can be identified on its own. For example, in a library database, a book is a strong entity because it has its own unique identifier, such as an ISBN number, and can exist independently of other entities.
- **Weak entity:** A weak entity is an entity that cannot be identified on its own and depends on another entity for identification. It does not have its own unique identifier and is identified by its relationship with another entity. For example, in a university database, a course offering may be a weak entity because it cannot exist without being associated with a course.
- **Associative entity:** An associative entity is an entity that is used to represent a relationship between two or more other entities. It contains attributes that describe the relationship between the other entities. For example, in a database for a music store, an associative entity could be used to represent the relationship between a customer and the songs they have purchased.

Relationship Sets

4.2 Types of Relationship Sets-

On the basis of degree of a relationship set, a relationship set can be classified into the following types-



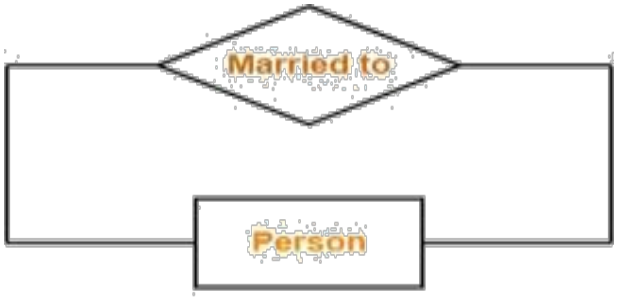
1) Unary relationship set
2) Binary relationship set
3) Ternary relationship set
4) N-ary relationship set

1. Unary Relationship Set-

Unary relationship set is a relationship set where only one entity set participates in a relationship set.

Example-

One person is married to only one person



Unary Relationship Set

2. Binary Relationship Set-

Binary relationship set is a relationship set where two entity sets participate in a relationship set.

Example-

Student is enrolled in a Course

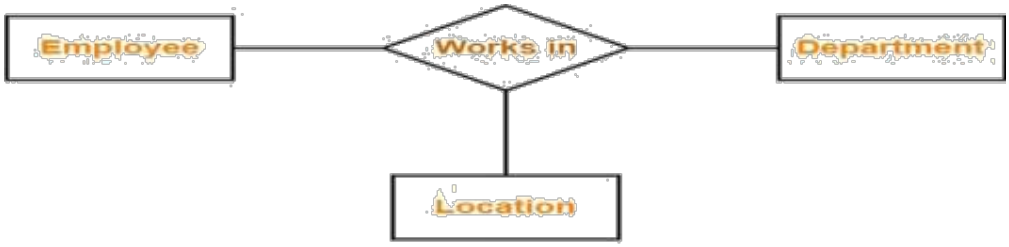


Binary Relationship Set

3. Ternary Relationship Set-

Ternary relationship set is a relationship set where three entity sets participate in a relationship set.

Example-



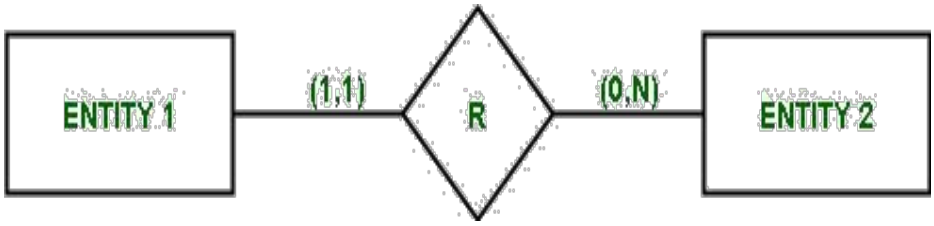
Ternary Relationship Set

4. N-ary Relationship Set-

N-ary relationship set is a relationship set where 'n' entity sets participate in a relationship set.

5.1 Roles and Structural Constraints

Structural Constraints are also called Structural properties of a database management system (DBMS). **Cardinality Ratios and Participation Constraints taken together are called Structural Constraints.** The name constraints refer to the fact that such limitations must be imposed on the data, for the DBMS system to be consistent with the requirements.



The Structural constraints are represented by Min-Max notation. This is a pair of numbers(m, n) that appear on the connecting line between the entities and their relationships. The minimum number of times an entity can appear in a relation is represented by m whereas, the maximum time it is available is denoted by n. If m is 0 it signifies that the entity is participating in the

A relationship is a **connection between two or more entities.** It describes how the entities are related or connected to each other. A relationship set is a collection of relationships of the same type. A relationship set is represented in an E-R diagram by a diamond, which is linked via lines to a number of different entity sets (rectangles).

For Example : An employee works for an organization. Here "works for" is a relation between the entities employee and organization.

However in ER Modeling, To connect a weak Entity with others, you should use a weak relationship notation as given below: -

Degree of a Relationship

Degree of a relationship is the number of entity types involved. The n-ary relationship is the general form for degree n. Special cases are unary, binary, and ternary ,where the degree is 1, 2, and 3, respectively.

Example for unary relationship : An employee ia a manager of another employee

Example for binary relationship : An employee works-for department.

Example for ternary relationship : customer purchase item from a shop keeper

Cardinality of a Relationship

Relationship cardinalities specify how many of each entity type is allowed. Relationships can have four possible connectivities as given below.

1. One to one (1:1) relationship

2. One to many (1:N) relationship

3. Many to one (M:1) relationship

4. Many to many (M:N) relationship

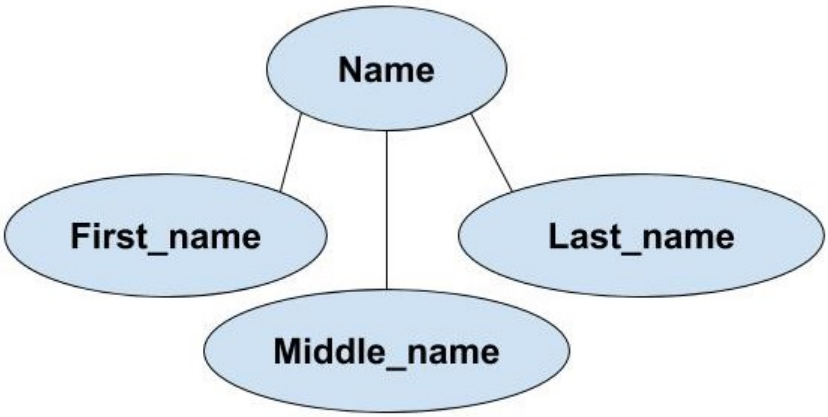
The minimum and maximum values of this connectivity is called the cardinality of the relationship.

Attributes

An attribute is a characteristic or property of an entity or relationship. For example, a person entity may have attributes such as name, date of birth, and address. In ER modeling, notation for attribute is given below.

1. **Key Attribute** :Key attribute uniquely identifies an entity from an entity set.

2. **Composite Attribute** :- An attribute that is composed of several other attributes is known as a composite attribute.



3. **Multivalued Attribute** : -Some attributes can possess over one value, those attributes are called multivalued attributes.

4. **Derived Attribute** : -An attribute that can be derived from other attributes of the entity is known as a derived attribute.

Constraints

constraints are rules or conditions that restrict the values that can be assigned to attributes or relationships between entities.

Mapping Cardinalities

Mapping cardinalities, or cardinality ratios, express the number of entities to which another entity can be associated via a relationship set. The mapping cardinality must be one of the following:

1. **One-to-One (1:1):** In a one-to-one relationship, each record in the first table is linked to only one record in the second table, and vice versa.

   For example, each employee can have only one office, and each office can be assigned to only one employee.

2. One-to-Many (1:M): In a one-to-many relationship, each record in the first table can be linked to many records in the second table, but each record in the second table can only be linked to one record in the first table.

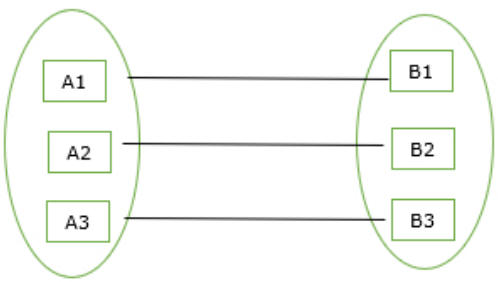
For example, each customer can have many orders, but each order can only belong to one customer.

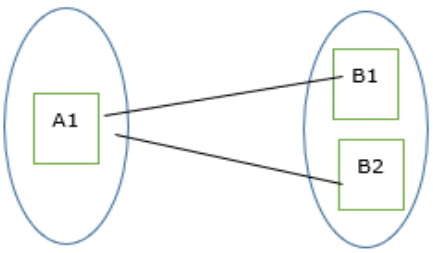
3. Many-to-one (M:1): When more than one element of an entity is related to a single element of another entity, then it is called a many-to-one relationship.

For example, students have to opt for a single course, but a course can have many students.

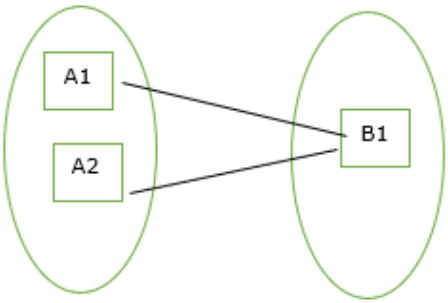
4. Many-to-Many (M:M): In a many-to-many relationship, each record in the first table can be linked to many records in the second table, and vice versa.

For example, each student can enroll in many courses, and each course can have many students.



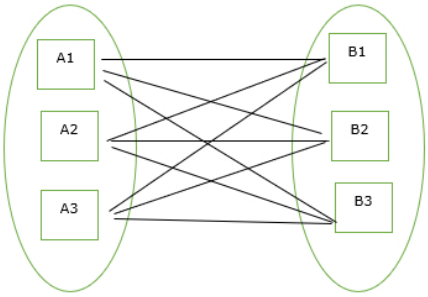


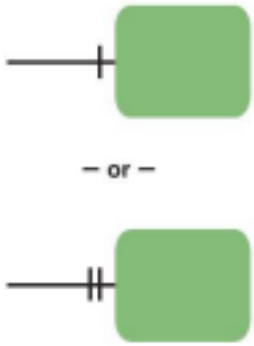
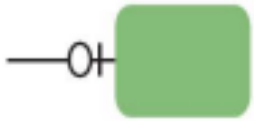
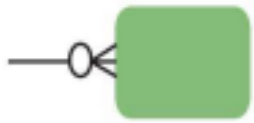
One-to-One (1:1)



Many-to-one (M:1)

One-to-Many (1:N)



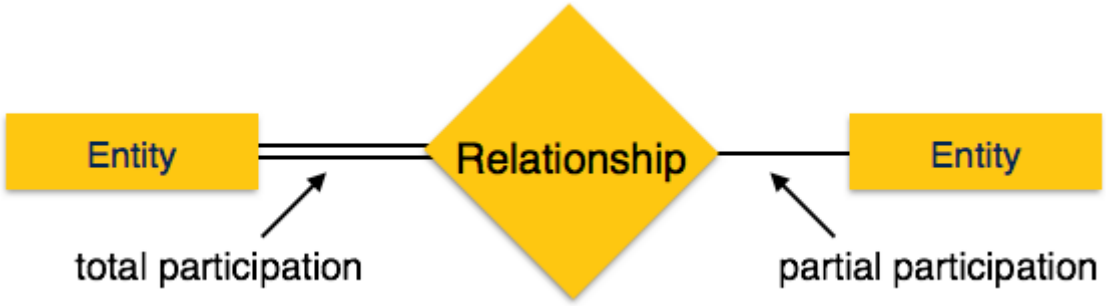
CARDINALITY INTERPRETATION	MINIMUM INSTANCES	MAXIMUM INSTANCES	GRAPHIC NOTATION
Exactly one (one and only one)	1	1	
Zero or one	0	1	
One or more	1	many (>1)	
Zero, one, or more	0	many (>1)	
More than one	>1	>1	

Participation constraints

The participation constraint specifies the number of instances of an entity can participate in a relationship set. In other words, they specify the degree to which an entity must participate in a relationship.

1.Total participation: In a total participation constraint, each instance of an entity must participate in a relationship. Each entity in the entity set is involved in at least one relationship in a relationship set i.e. the number of relationship in every entity is involved is greater than 0. For example, in a university database, each student must be enrolled in at least one course, so the Student entity has a total participation constraint in the Enroll relationship.

2. Partial participation: In a partial participation constraint, some instances of an entity may not participate in a relationship. Each entity in entity set may or may not occur in at least one relationship in a relationship set. For example, in a university database, a course may not have any students enrolled yet, so the Course entity has a partial participation constraint in the Enroll relationship.



Keys

keys represent attributes or sets of attributes that uniquely identify an entity or relationship. Keys are important in database design because they ensure data integrity and help to establish relationships between entities. Types of keys:-

1.Primary key: A primary key is an attribute or set of attributes that uniquely identifies an entity in a database. Each entity must have a primary key, which is denoted by underlining the attribute(s) in the ER diagram.

2. Super key: A super key is an attribute or set of attributes that uniquely identifies an entity, but may also contain additional non-essential attributes. It is not always the same as the primary key.

3.Candidate key: A candidate key is a minimal set of attributes that can uniquely identify an entity. It is a subset of the super key, and can be used as an alternative primary key.

4. Foreign key: A foreign key is an attribute or set of attributes in one entity that refers to the primary key of another entity. It is used to establish relationships between entities in a database.

Entity Relationship Diagram

An entity-relationship diagram (ERD) is a graphical representation of entities and their relationships to each other.

Basic Structure

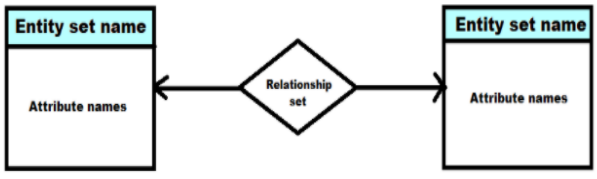
- Rectangles: These are classified into two parts. The first half coloured in blue contains the name of the entity set. The second half in white contains the names of all the attributes. The attributes that are part of the key are underlined.
- Diamonds: These represent relationship sets.
- Undivided Rectangles: These represent the attributes present in a relationship set.
- Lines: The lines help us to link entity sets to relationship sets.
- Dashed Lines: These help link attributes of a relationship set to another relationship set.
- Double Lines: These indicate the total participation of an entity in a relationship set.
- Double Diamond: These represent identifying relationship sets that are linked to weak entity sets.
- E-R diagrams show a way to point out complex constraints on the number of times every entity takes part in the relationships in a relationship set.

OR, **Basic Structure**

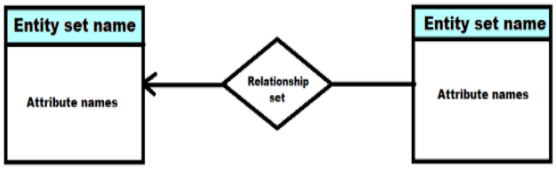
- Entities: These are the main objects or concepts in the system that are represented as rectangles.
- Attributes: These are the characteristics or properties of the entities that are represented as ovals or circles.
- Relationships: These are the associations or connections between the entities that are represented as diamond shapes.
- Cardinality: The cardinality of a relationship is represented by a line connecting the diamond shape to the entity rectangle with a cardinality symbol written on it.
- Keys: These are used to uniquely identify each entity instance.
    - Primary Key: A primary key is represented by an underline under the attribute name.
    - Foreign Key: A foreign key is represented by a dashed underline under the attribute name.

Mapping cardinalities in E-R diagram

- One-to-One Mapping Cardinalities

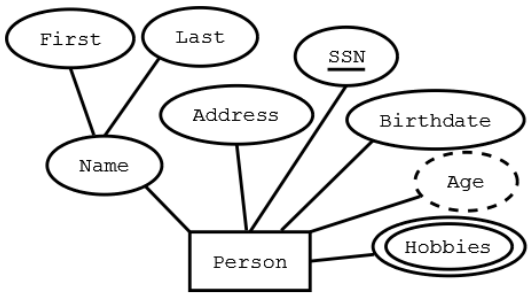


- One-to-Many Mapping Cardinalities



Complex Attributes

The attributes that are formed by arbitrarily nesting the composite and multivalued attributes are called complex attributes.



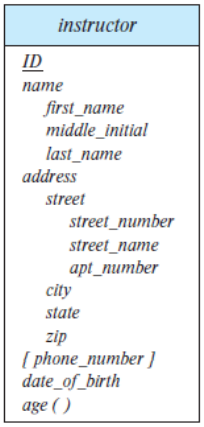
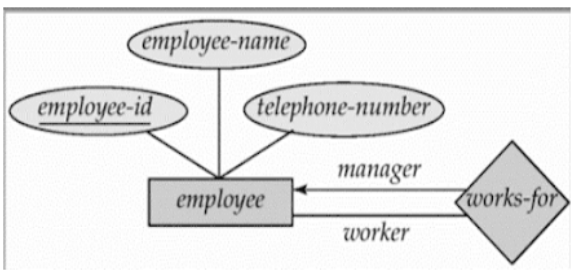
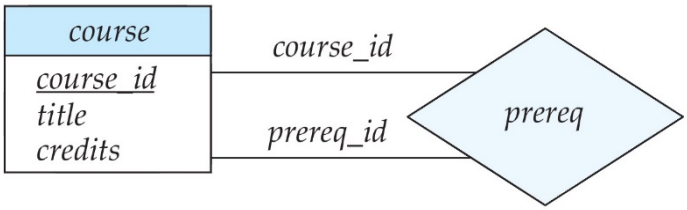


Figure 6.8 E-R diagram with composite, multivalued, and derived attributes.

Roles

Roles are used to specify the different functions or responsibilities of an entity in a relationship. They describe the way that two entities are related to each other and their specific roles or positions in that relationship. Roles are indicated in E-R diagrams by labeling the lines that connect diamonds to rectangles.



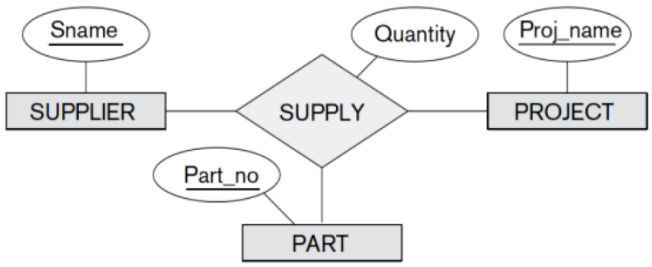


For Example, The relationship works-for define in employees as manager or worker. The labels "manager" and "worker" are called roles; they specify how employee entities interact via the works-for relationship set.

Nonbinary Relationship sets

A nonbinary relationship set is a relationship between more than two entities that is non-binary relationship. A binary relationship is a relationship between two entities. A nonbinary relationship can be a ternary relationship, a quaternary relationship, or a relationship between any number of entities.

For example, a relationship between a student, an instructor, and a course would be a ternary relationship.

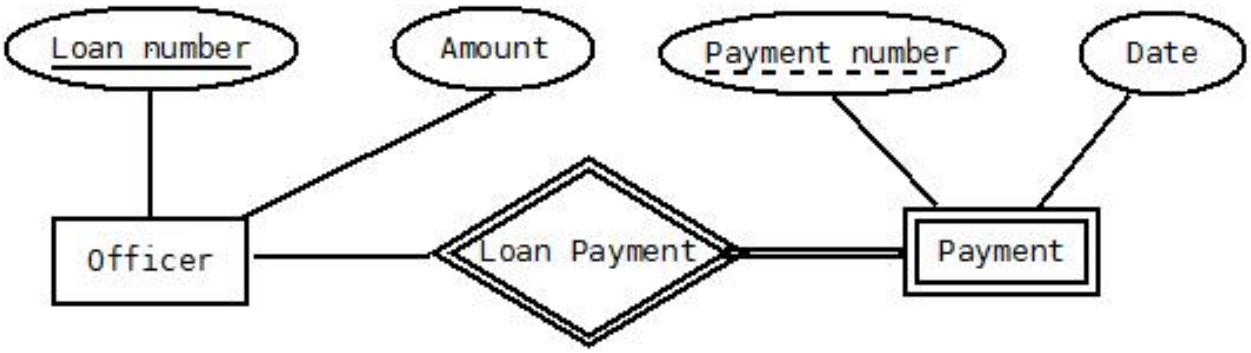


Weak Entity Sets

A weak entity set is an entity set that cannot be uniquely identified by its attributes alone. It depends on the existence of a related entity set, known as the identifying entity set, for its identification.

There are two main components of a weak entity set: the weak entity and the identifying relationship.

- Weak Entity :- A weak entity is represented by a double rectangle in an ER diagram.
- Identifying Relationship :- It connects the weak entity set to the identifying entity set and is represented by a diamond-shaped symbol on the identifying entity set side.



Strong Entity Set

A strong entity set is an entity set that has a primary key attribute, which uniquely identifies each entity in the set. A strong entity can exist independently of any other entity in the database.

Key characteristics of a strong entity set:

- Unique Identifier: A strong entity set has its own unique identifier or primary key attribute that distinguishes each instance of the entity set. The primary key attribute ensures that each entity within the set can be uniquely identified.
- Independent Existence: A strong entity set can exist independently in the database system. It does not rely on any other entity set for its existence or identification.
- Relationship Participation: A strong entity set can participate in relationships with other entity sets in the ER diagram. It can be connected to other entity sets through relationships, representing associations or associations between entities.

A strong entity set is represented by a rectangle, and its attributes are listed within the rectangle. The primary key attribute is typically underlined or highlighted to indicate its significance.

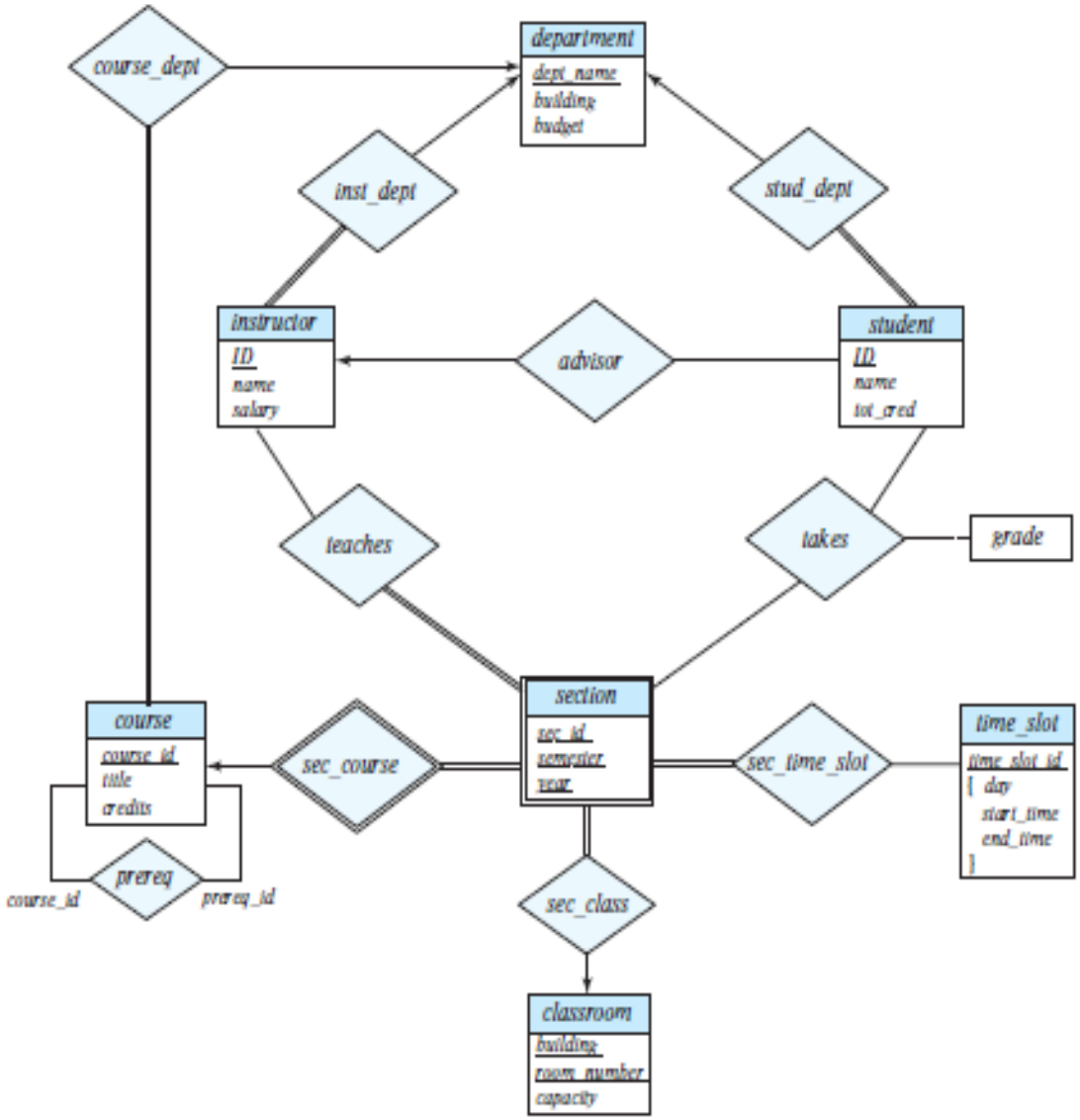


Figure 6.15 E-R diagram for a university enterprise.

Extended E-R Features

The ER model with extended features **is an enhancement of the traditional ER model**, which includes additional modeling constructs to provide more expressiveness and flexibility in modeling complex data scenarios. It allow you to model hierarchical relationships between entities.

Specialization

It will inherit the attributes from main entity(superclass). Specialization can also be used to improve the security of a database. (Role based access). **An entity is divided into sub-entities(subclasses) based on their characteristics.** It is a top-down approach where higher level entity is specialized into two or more lower-level entities.

Specialization is the process of dividing an entity set into a set of more specialized entity sets.

For example, you could have an entity set called "Vehicle" that is specialized into the entity sets "Car", "Truck", and "Motorcycle".

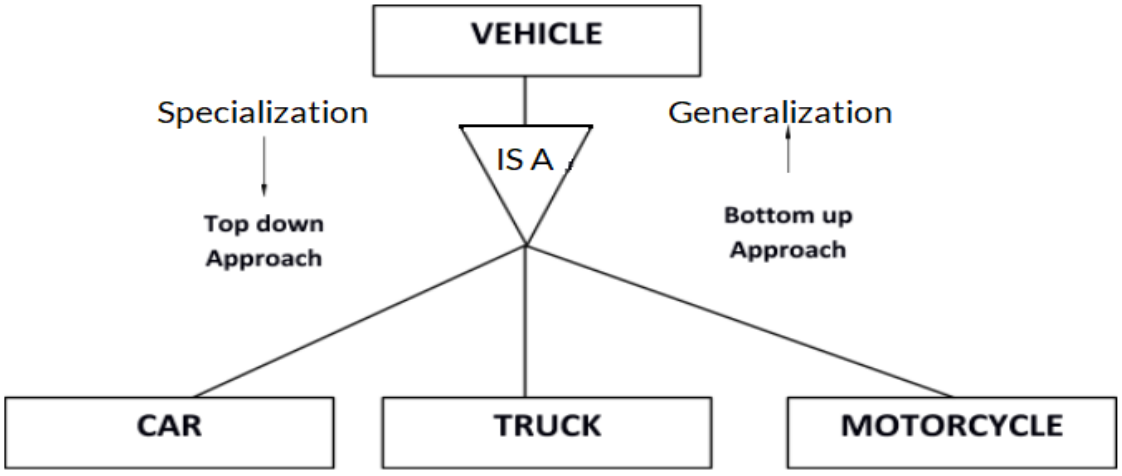
Generalization

It is the process of **combining two or more entity sets into a single**, more general entity set. Generalization is a Bottom-up process.

A set of entities that share common attributes, relationships, and behaviors are grouped together to form a higher-level entity, known as a superclass or generalization. The entities that are grouped together are called subclasses or specializations. The generalization hierarchy represents an "is-a" relationship, where the superclass is a more general category, and the subclasses are more specific types or variations of that category.

For example, a database might have entities for cars, trucks, and buses. These entities all have the attributes make, model, and year. Generalization can be used to create a single entity for vehicles.

Subclasses can share attributes with their superclass. In fact, this is one of the main benefits of using specialization. By sharing attributes, you can reduce the amount of data that needs to be stored in the database.



Constraints on Generalization

Constraints on Generalization specify the rules that need to be followed when converting a superclass entity to a subclass entity. One type of constraint involves determining which entities can be members of a given lower-level entity set. Such membership may be one of the following:

A **condition-based constraint** is a type of constraint that is based on a condition. if a condition-based constraint is applied to a generalization, then the database will not allow an entity to belong to a subclass if the condition is not met. This can help to prevent errors in data entry and to ensure that the data is consistent.

For example, you could create a condition-based constraint that specifies that an employee can only be a full-time employee if their salary is greater than $50,000.

User-defined. User-defined lower-level entity sets are not constrained by a membership condition; rather, the database user assigns entities to a given entity set.

For instance, let us assume that, after 3 months of employment, university employees are assigned to one of four work teams. We therefore represent the teams as four lower-level entity sets of the higher-level employee entity set. A given employee is not assigned to a specific team entity automatically on the basis of an explicit defining condition. Instead, the user in charge of this decision makes the team assignment on an individual basis. The assignment is implemented by an operation that adds an entity to an entity set.

A second type of constraint relates to whether or not entities may belong to more than one lower-level entity set within a single generalization. The lower level entity sets may be one of the following:

- **Disjoint** – The requirement of this constraint is that an entity should not belong to no more than one lower-level entity set.
    - For example, an employee can only be a full-time employee or a part-time employee, but not both.
- **Overlapping** – In this category of generalizations, within a single generalization, the same entity may belong to more than one lower-level entity set.
    - For example, a person can be a student and an employee at the same time.

A final constraint, the completeness constraint on a generalization or specialization, specifies whether or not an entity in the higher-level entity set must belong to at least one of the lower-level entity sets within the generalization/specialization.

This constraint may be one of the following:

• **Total generalization or specialization**. Each higher-level entity must belong to a lower-level entity set.

• **Partial generalization or specialization**. Some higher-level entities may not belong to any lower-level entity set

Aggregation

Aggregation is a modeling technique in which a relationship between entities is treated as a higher-level entity. In other words, it is a way to represent a relationship between entities as a single entity. Aggregation is an abstraction through which we can represent relationships as higher level entity sets. It allows for the modeling of complex relationships and the representation of higher-level objects composed of other objects.

Aggregation is an abstraction through which relationships are treated as higher level entities.

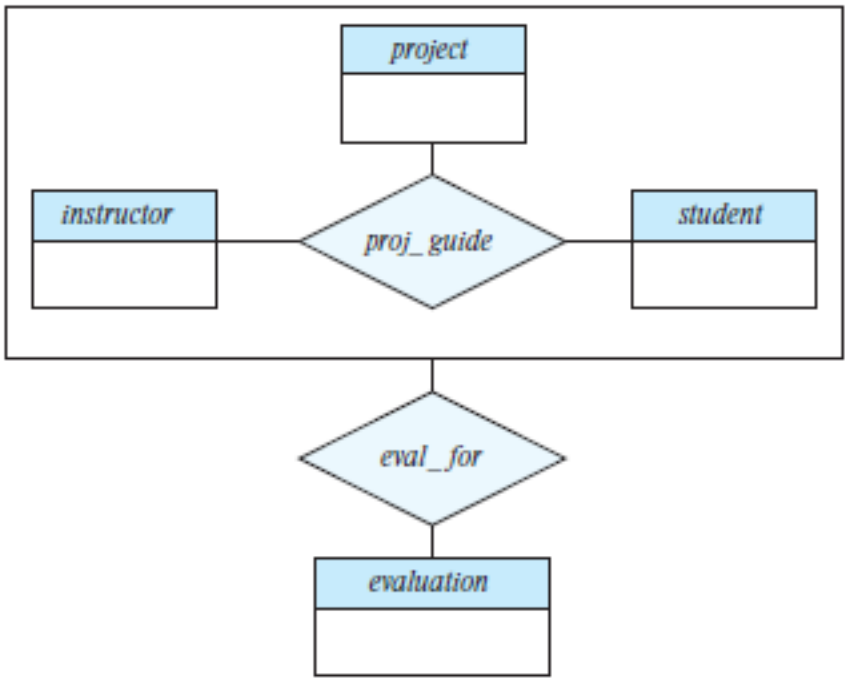


Figure 6.20 E-R diagram with aggregation.

E-R Notations For Modeling Data

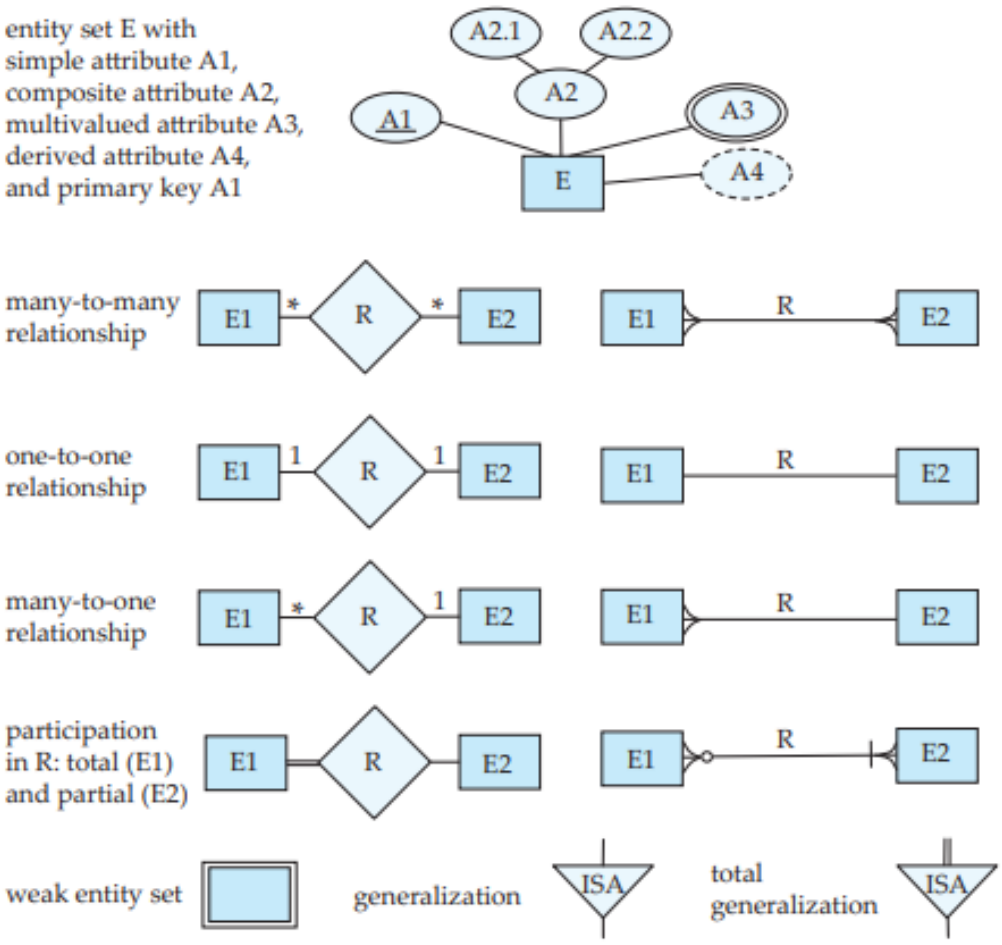


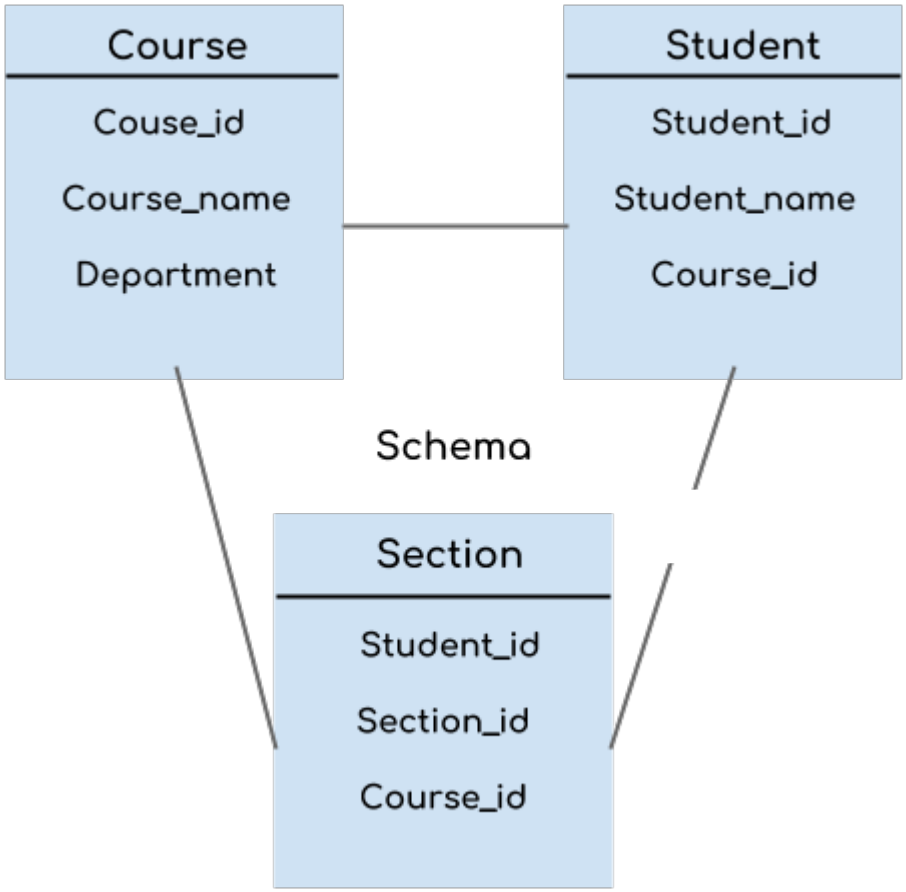
Figure 7.25 Alternative E-R notations.

Instance and schema in DBMS

DBMS Schema

Definition of schema: Design of a database is called the schema. Schema is of three types: Physical schema, logical schema and view schema.

For example: In the following diagram, we have a schema that shows the relationship between three tables: Course, Student and Section. The diagram only shows the design of the database, it doesn't show the data present in those tables. Schema is only a structural view(design) of a database as shown in the diagram below.



The design of a database at physical level is called **physical schema**, how the data stored in blocks of storage is described at this level.

Design of database at logical level is called **logical schema**, programmers and database administrators work at this level, at this level data can be described as certain types of data records gets stored in data structures, however the internal details such as implementation of data structure is hidden at this level (available at physical level).

Design of database at view level is called **view schema**. This generally describes end user interaction with database systems.

DBMS Instance

Definition of instance: The data stored in database at a particular moment of time is called instance of database. Database schema defines the variable declarations in tables that belong to a particular database; the value of these variables at a moment of time is called the instance of that database.

For example, lets say we have a single table student in the database, today the table has 100 records, so today the instance of the database has 100 records. Lets say we are going to add another 100 records in this table by tomorrow so the instance of database tomorrow will have 200 records in table. In short, at a particular moment the data stored in database is called the instance, that changes over time when we add or delete data from the database.

RELATIONAL MODEL

1. Relation Schema and Relation Instance

- **A relation schema** is a list of attribute names (user defined domain names), e.g. Account = (branch-name, account#, balance). However, for various reasons, a relation schema is often represented by a set (not an ordered list) of attributes.

- **A relation instance** is a subset of a Cartesian product of a list of domains.

  e.g., account(Account): "account" is a relation instance on "Account" schema. The relation instance "account" is a subset of D1 $\times$ D2 $\times$ D3, where D1, D2, and D3 are the domains of "branch-name", "account-name", and "balance", respectively.

- **A tuple** is an element of a relation instance. If there are n attributes, each tuple of the relation instance is an ordered n-tuple. Thus, each tuple is an n-ary relationship between n attribute values. A relation is, in fact, a set of such relationships.

- **"relation" often means "relation instance"**

- **"database" often means "database instance"**

- **Recall the three-level database (Lecture Note#1). In the logical level, database schema is a set of relation schemas and database instance is a set of relation instances.**

Note, in relational data model, relation schema consists of atomic domains. A domain is atomic if each element of the domain is conceptually indivisible. The balance of an account is atomic. However, the domains of "date" or "author-list" of a book are not atomic.

c.f. Object-Relational DB allows complex data types (i.e., non-atomic domains).

- **Keys:** Superkey, Candidate key, Primary key, and Foreign key.

  Candidate key: a set of attributes whose values uniquely identify individual tuples in the relation (e.g., {Social Security#}, {Employee#}, {Employee#, Dependent Name}). None of its subset can be used to uniquely identify tuples (e.g., {Employee#} or {Dependent Name} cannot uniquely identify a tuple in the "Dependent" table). "Null" value is not allowed. Each tuple has a unique candidate key value in the relation.

Primary key: one of the candidate keys of the relation, chosen by the designer. "Null" value is not allowed (i.e., all attributes of the primary key are implicitly declared to be not null). Each tuple has a unique primary key value in the relation. If an attribute set P is the primary key of relation r, then P must be a candidate key of r. The opposite does not hold

Super key: union of (one or more candidate keys) and (zero, one, or more extraneous attributes). If an attribute set C is a candidate key of relation r, then C must be a super key of r. The opposite does not hold.

Foreign key: We use this key to represent the relationship between relations. Let r(R) and s(S) be relations with primary keys Kr $\subseteq$ R and Ks $\subseteq$ S, respectively. If Fr $\subseteq$ R is a foreign key referencing Ks, for every t1 $\in$ r with t1[Fr] $\neq$ null, there exist a tuple t2 $\in$ s such that t1[Fr] = t2[Ks]. Note, the mapping cardinality between r and s must be one-to-one or many-to-one (note, r is on the many side). Note, t1[Fr] is the foreign key value of tuple t1 and t2[Ks] is the primary key value of tuple t2.

- **A Query Language is** a language in which users request information from the underlying database. (Relational Algebra, Relational Calculus, QBE, SQL, …)

E-R Diagram: Subclass, E-R to relational design

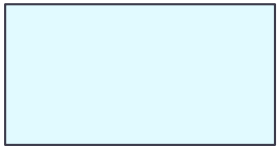
CS 4750 Database Systems

[A. Silberschatz, H. F. Korth, S. Sudarshan, Database System Concepts, Ch.6]
[C.M. Ricardo and S.D. Urban, Database Illuminated, Ch.3]

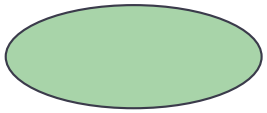
revisit

E-R Diagram: Building Blocks

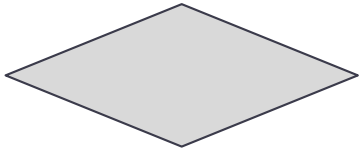
(strong) Entity set



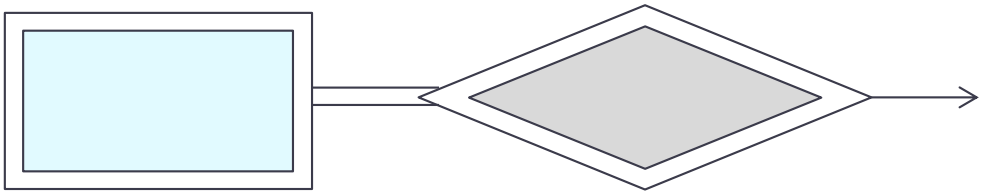
Attribute



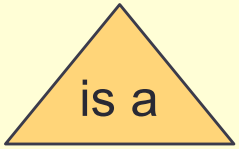
Relationship



Weak entity



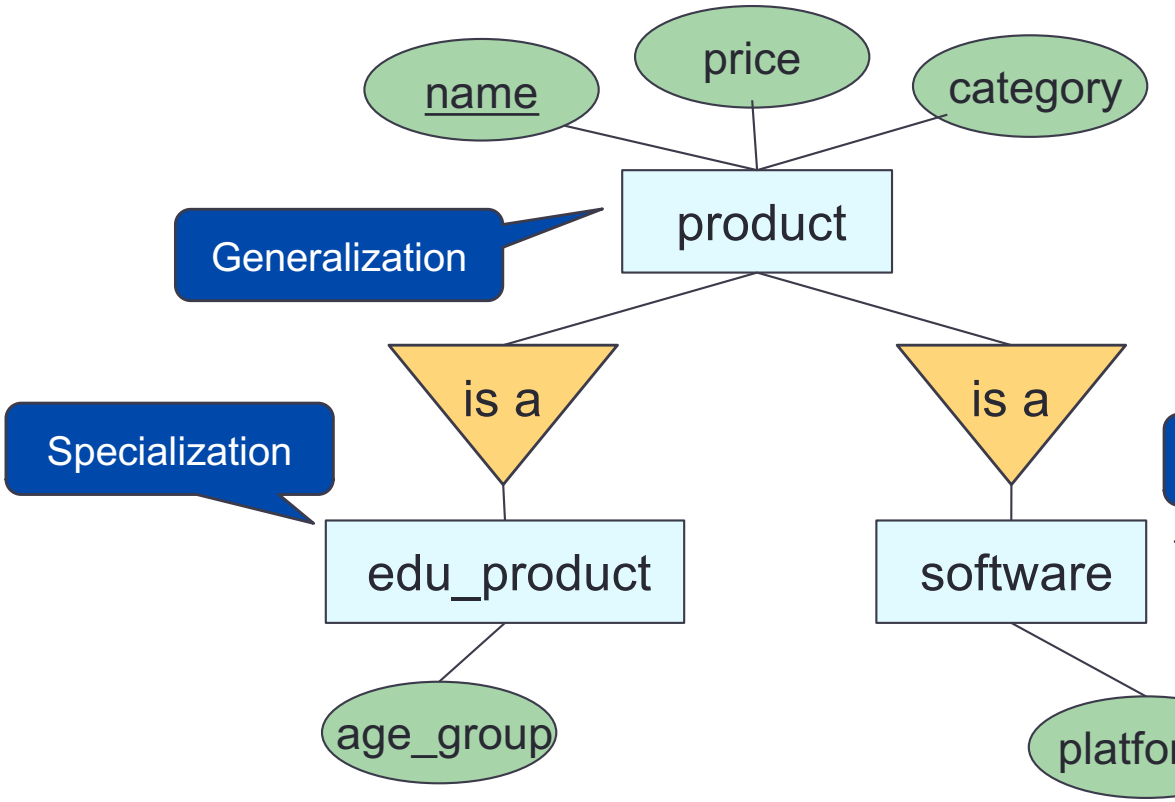
Subclass



Note: colors are not part of E-R Diagram. They simply are used to increase readability.

Subclassing

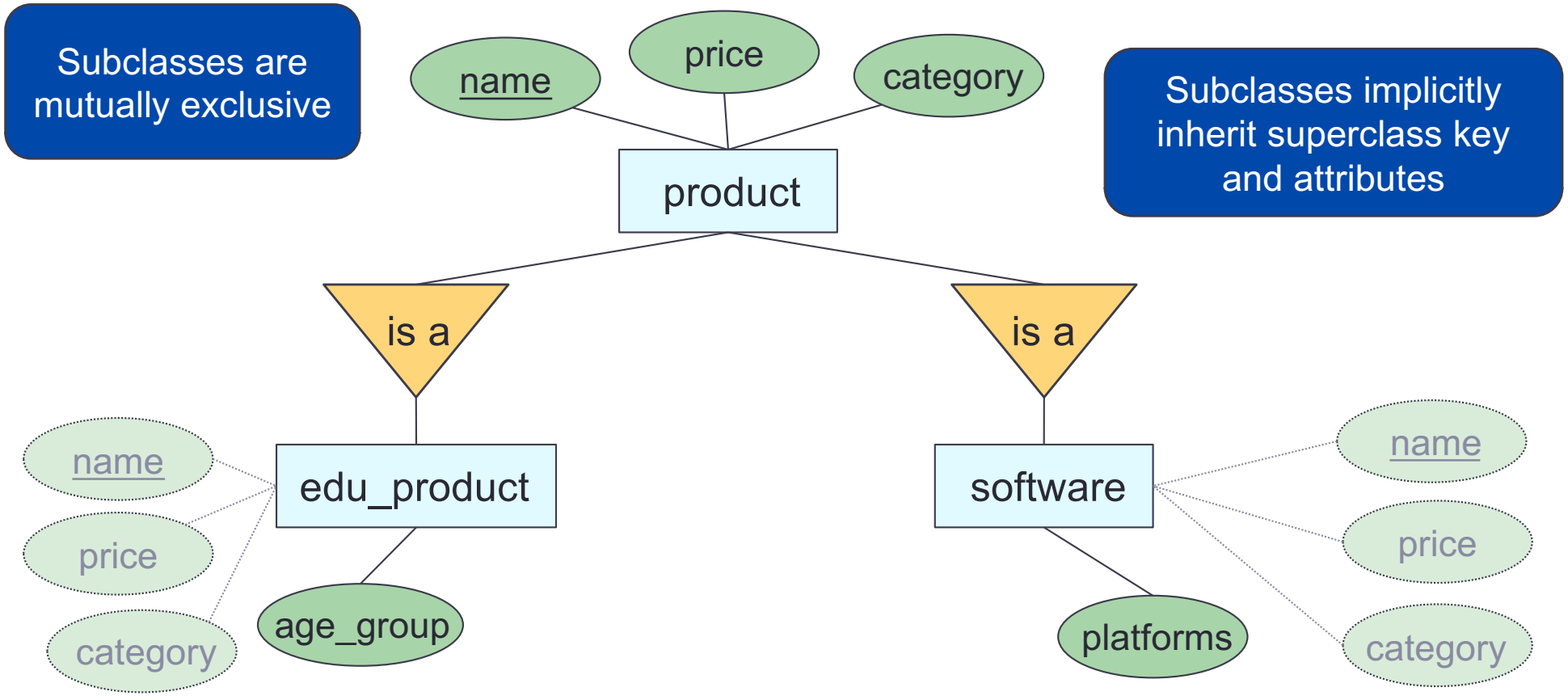
- An entity set may contain entities that have special properties not associated with all members of the set
- Subclasses ~ special-case entity sets
- Isa (or Is-a) ~ special kind of relationship (one-to-one)



Similar to classes in OO (super-classes and sub-classes)

The triangle points to the specialization

Subclassing



Generalization / Specialization
The triangle points to the specialization

Decisions to Make

- Entity set vs. attributes
  - Has more data → entity set
  - Is the data → attribute

- Entity set vs. relationship set
  - Entity set → nouns (students, faculty, loads, …)
  - Relationship → possession verbs (teaches, advises, owns, works for, …)

- Binary vs. n-ary relationship sets
- Specialization / generalization

Rules of Thumb

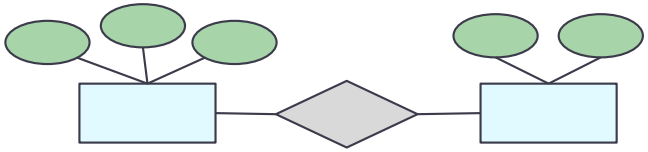
- Pick the right entities
- Keep it simple
- Don’t over complicate things
- Choose the right elements (entities vs. attributes)
- Choose the right relationships
- Follow the specification of the application to be built
- Avoid NULL value
- Avoid redundancy
- Consider small number of tables

Database Design Process

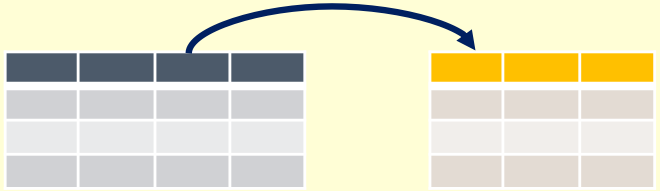
revisit

Interact with users and domain experts to characterize the data

Translate requirements into conceptual model (E-R diagrams)



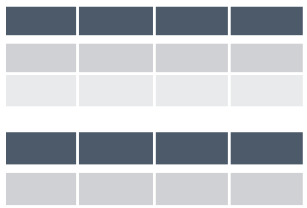
Convert the model to relational model (schema and constraints)

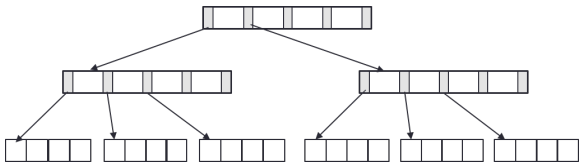


Normalize and develop conceptual (logical) schema of the database



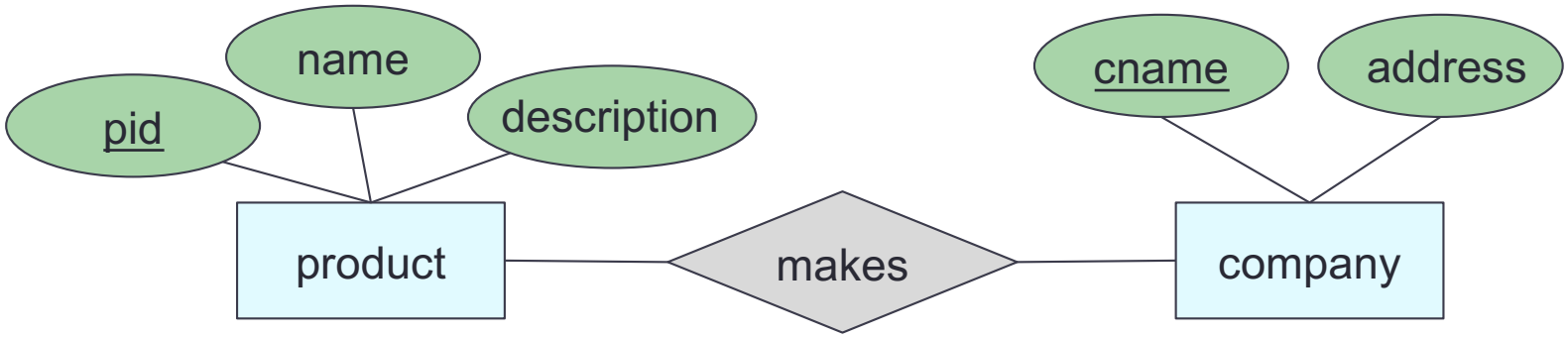
Develop physical schema (partitioning and indexing)





E-R Diagrams to Relations

There is a unique table which is assigned the name of the corresponding entity set or relationship set



product(pid, name, description)

company(cname, address)

makes(cname, pid)

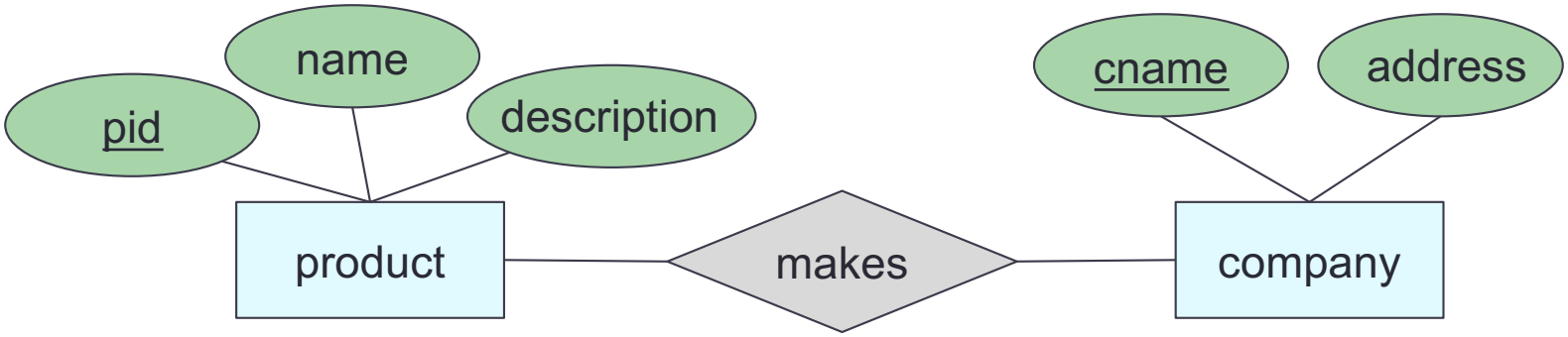
"Schema statement"

[ER to schema worksheet]

Strong Entity Set

Direct map:

- Entity name → relation name
- Attributes → columns
- Primary key: same as entity

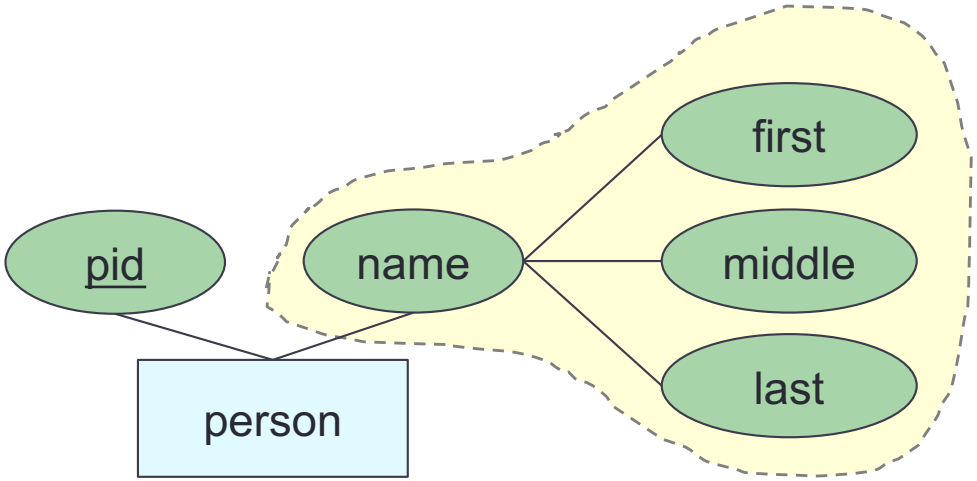


product(<u>pid</u>, name, description)
company(<u>cname</u>, address)
makes(<u>cname</u>, <u>pid</u>)

Strong Entity Set with Composite Attribute

Create separate attributes for each component

Don’t include the higher level attribute



person(<u>pid</u>, first_name, middle_name, last_name)

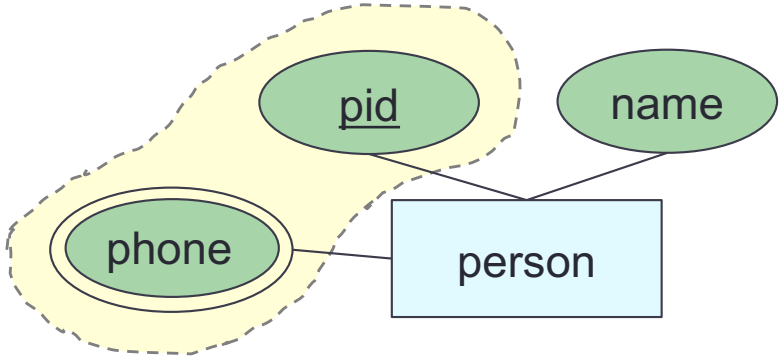
[ER to schema worksheet]

Strong Entity Set with Multivalued Attribute

Create a separate table for the multivalued attribute

Name the table with the concatenation, separated by "_"
entityname_attributename

Primary key: all attributes



person(pid, name)

person_phone(pid, phone_number)

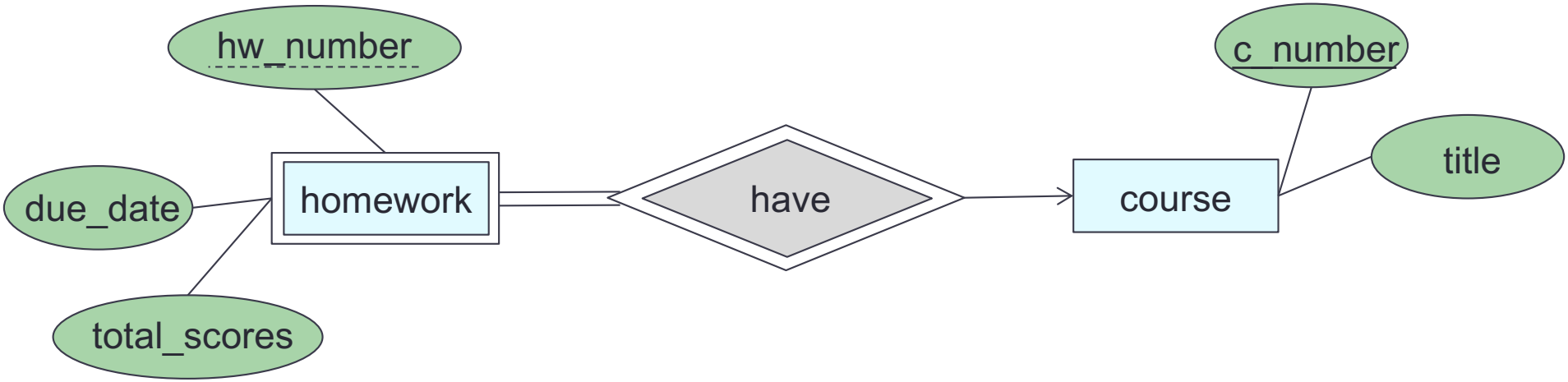
[ER to schema worksheet]

Weak Entity Set

Let *A* be a weak entity set and *B* be the identifying strong entity set on which *A* depends

Create a table with primary key of *B* and all *A*'s attributes

Primary key: primary key of *B* (strong entity) and
discriminator of *A*



course(c_number, title)

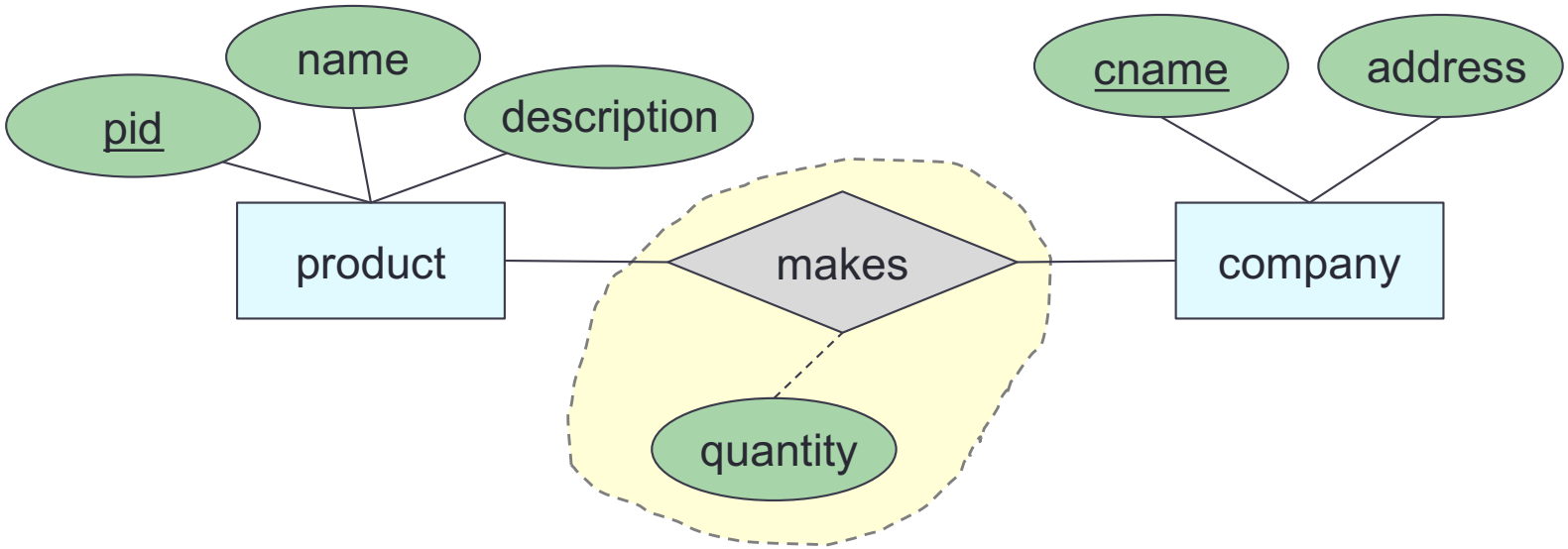
homework(c_number, hw_number, due_date, total_scores)

[ER to schema worksheet]

Relationship Set: Many-to-Many

Table: primary keys of both participating entity sets and any attributes on the relationship itself

Primary key: primary keys of both participating entity sets



product(pid, name, description)
company(cname, address)
makes(pid, cname, quantity)

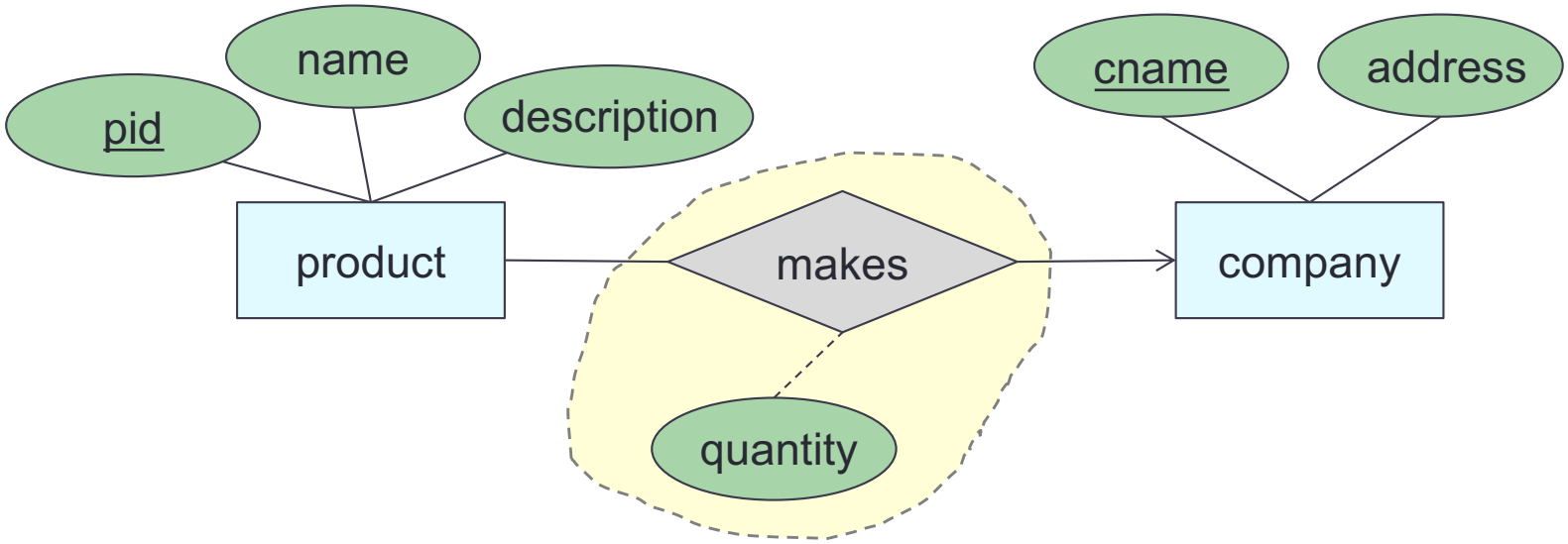
Primary keys of both entities

[ER to schema worksheet]

Relationship Set: Many-to-One / One-to-Many

Table: primary keys of both participating entity sets and any attributes on the relationship itself

Primary key: primary keys of the entity set on the "many" side



product(pid, name, description)
company(cname, address)
makes(pid, cname, quantity)

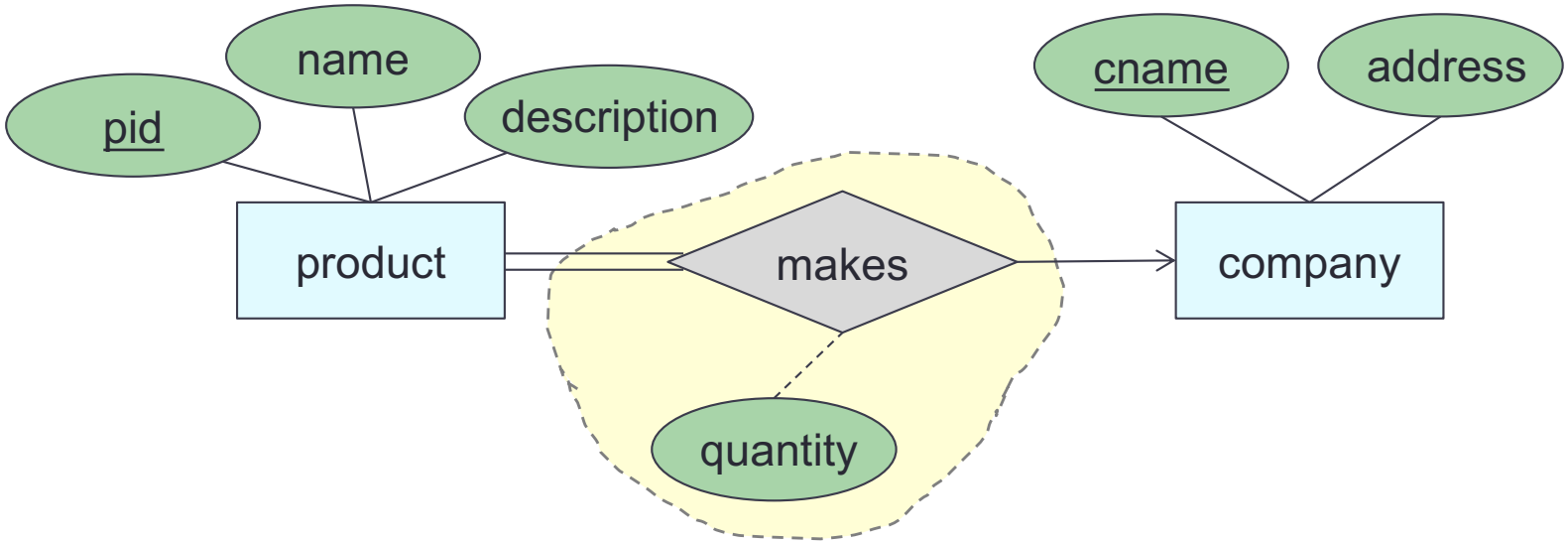
Primary key of the "many" side

[ER to schema worksheet]

Relationship Set: Total Participation

Because the total participation requires all entity to be participated in the relationship

→ add the primary key of the "one" side to the "many" side entity set, no table for relationship needed



product(pid, name, description, cname, quantity)
company(cname, address)

[ER to schema worksheet]

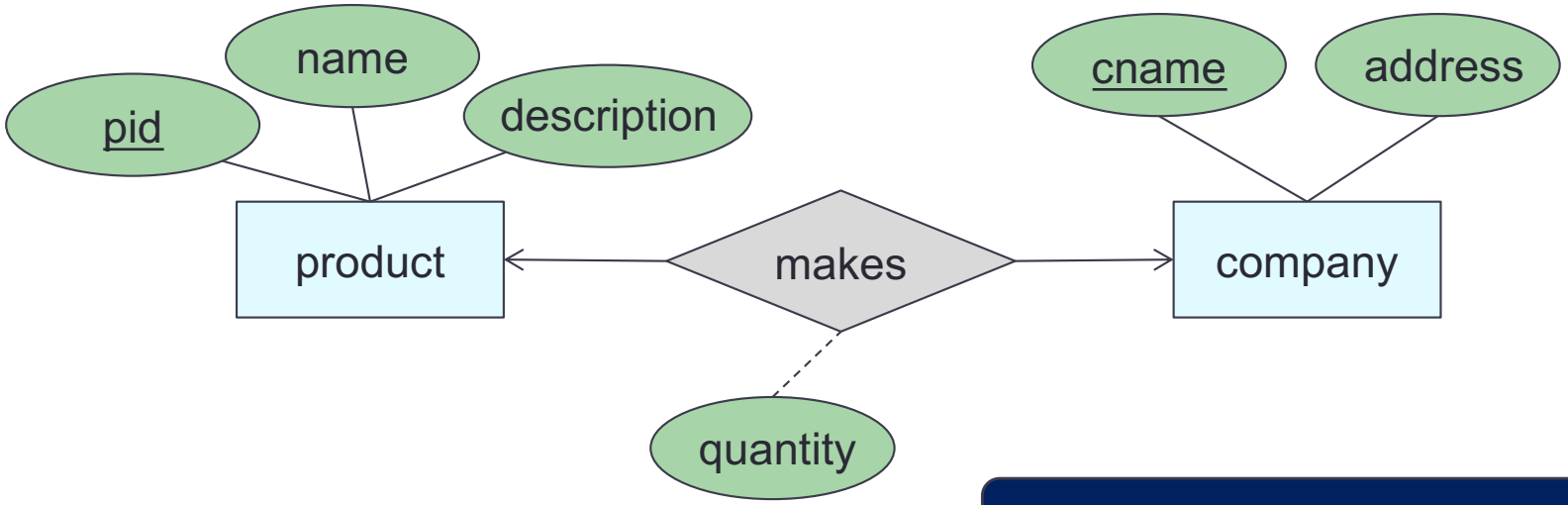
Relationship Set: One-to-One

Table: Either side can be used as the main table

(Which side? doesn't matter. Pick the one that makes the most sense)

Add the other side's primary key to it

Primary key: primary keys of the entity set you pick

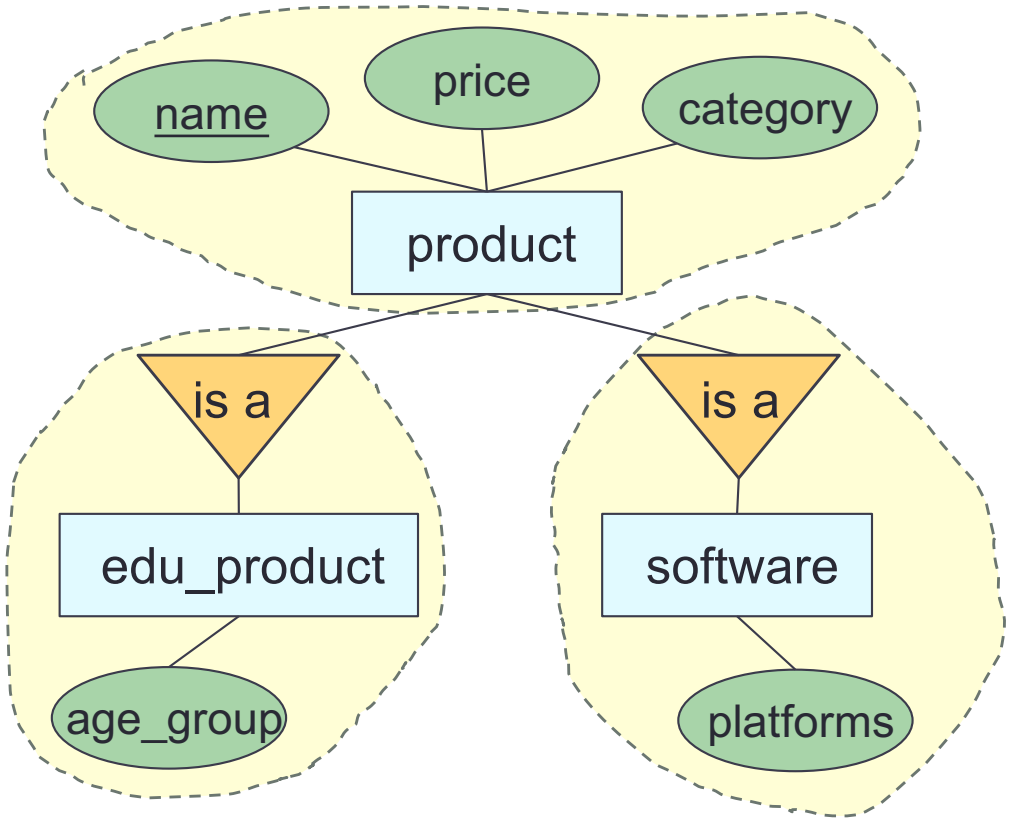


Primary key of the chosen entity

product(pid, name, description)
company(cname, address, pid, quantity)

[ER to schema worksheet]

Subclass (Option 1)



Keep everything

Primary key of the lower level entity set: from the higher level

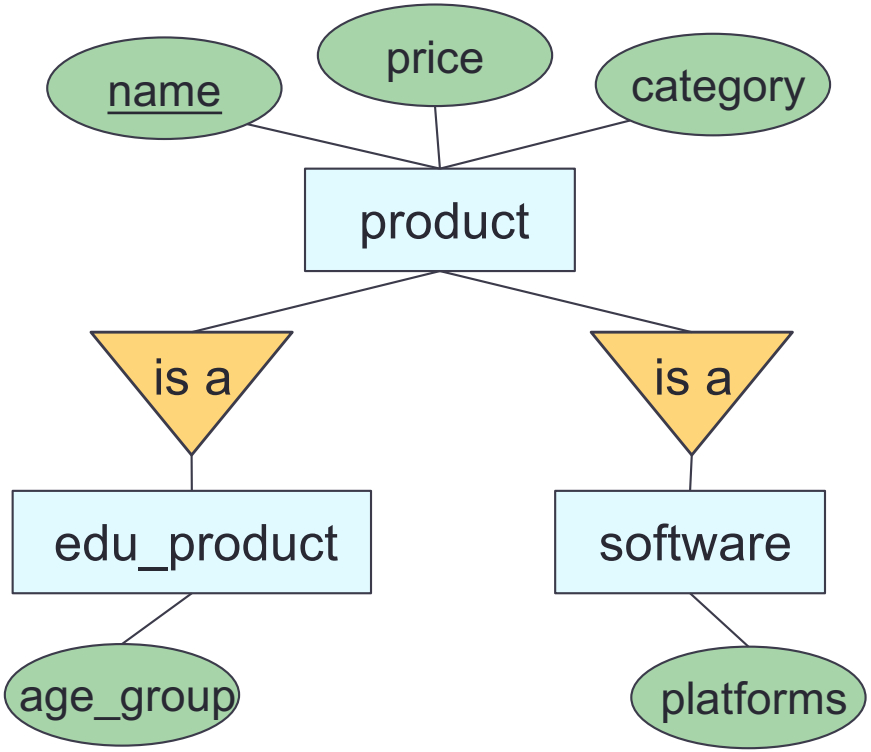
Drawback: need to access more tables to get info about the lower levels

product(<u>name</u>, price, category)
edu_product(<u>name</u>, age_group)
software(<u>name</u>, platforms)

Keep everything

[ER to schema worksheet]

Subclass (Option 2)



Keep specialization entity sets

No table for generalization entity set

Primary key of the lower level entity set: from the higher level

Drawback: redundancy if entities have more than one specialization

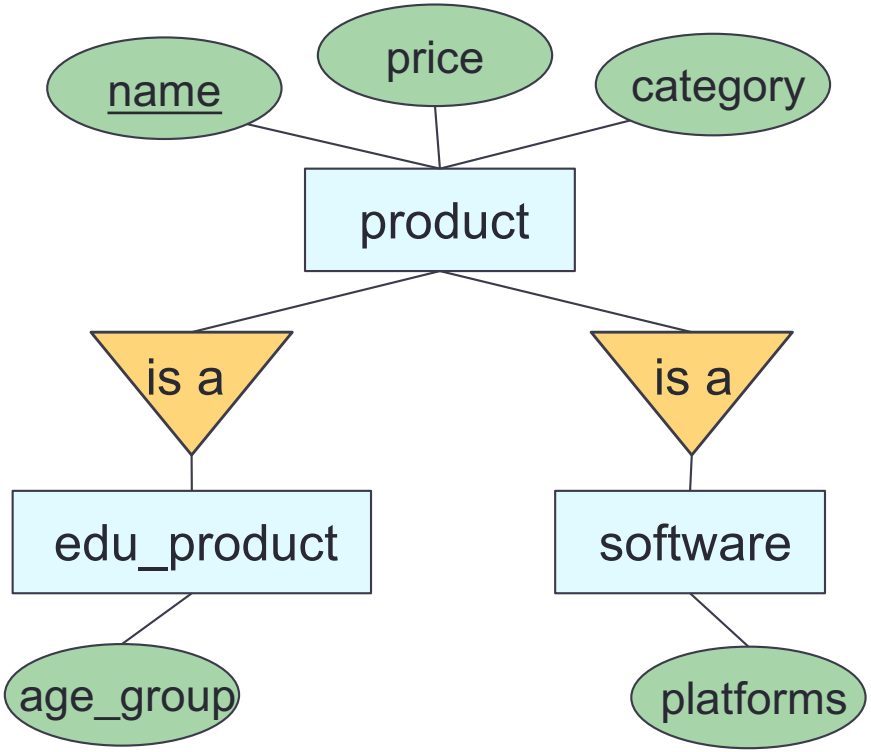
edu_product(name, price, category, age_group)

software(name, price, category, platforms)

Push down

[ER to schema worksheet]

Subclass (Option 3)



Keep generalization entity set

No table for specialization entity sets

Drawback: NULL in attributes from specialization entity sets

Although less duplication of data, need to handle NULL value

product(name, price, category, age_group, platforms)

Push up

[ER to schema worksheet]

Subclass: Design Decision

Depending on the number of attributes of the generalization entity set and specialization entity set

- If balanced → do option 1 (create all)
- If more attributes in specialization → do option 2
- If more attributes in generalization → do option 3

In general, design decision depends on

- The number of attributes
- DB administrator's decision

Overall goal: minimize duplication
(there is no one correct way)

Wrap-Up

- Roles in Relationships
- Relationships: binary, n-ary
- Weak entity
- Subclasses
- Converting from E-R diagrams to relational designs
  - Turn each entity set into a relation with the the same set of attributes
  - Replace a relationship by a relation whose attributes are the keys for the connected entity sets
  - Weak entity sets cannot be translated straightforwardly to relations
  - "Is a" relationships and subclasses require careful treatment

What's next?

- Apply the concept to database scenarios and fine-tuning database structure

MAPPING ER DIAGRAMS TO RELATION DATA MODEL

- Conceptual ER-models allow you to more accurately represent the subject area than logical models (relational, network, etc.).
- But, there are no DBMSs that support ER models.
- So, ER diagram is converted into the tables in Relational Data Model (RDM or RM).
- Relational models can be easily implemented by RDBMS like mySQL, MS SQL, PostgreSQL, Oracle etc. With CASE Tools usage.

The ER2RDM mapping method is based on the formation of a set of initial relation tables from ER-diagrams (initial logical model) and based on this factors – **atomic and multivalued** of attribute, **cardinality** (max=one-many) **and obligation** (min=optional-mandatory) of relationship.

At the next stage, the initial logical RD model are optimized (normalized).

1. ER2RDM MAPPING BRIEFLY

- A table is created for each entity:
  01) Each simple entity attribute corresponds to **a current table** column, derived entity attribute removed from **a current table**;
  02) Each element of composite attribute corresponds to **a current table** column;
  03) Each multivalued attribute **+1 separate table** will require.
- For Binary Relationship With Weak Entity Set:
  04) – **2 tables** for 2 entity will require.
- For a relationship without cardinality ratios description:
  05) – **3 tables** will require.
- For a relationship of type m:n, there are only 1 rule:
  06) relationship n:m – **3 tables** for 2 entity will require.
- For type 1:m relationship, there are 2 separate rules:
  07) relationship 1,1:1,m or 0,1:1,m – **2 tables** for 2 entity will require;
  08) relationship 1,1:0,m or 0,1:0,m – **3 tables** for 2 entity will require.
- For a 1:1 relationship, there are 3 separate rules for generating preliminary tables from ER diagrams; (min,max:min,max) notation use:
  09) relationship 1,1:1,1 – **1 table** for 2 entity will require (Employee -||---||- Badge => EmployeeBadge);
  10) relationship 1,1:0,1 or 0,1:1,1 – **2 tables** for 2 entity will require (Manager -||---O|- Branch => Manager + Branch);
  11) relationship 0,1:0,1 – **3 tables** for 2 entity will require (Manager -|O---O|- Branch => Manager + ManagerBranch + Branch).

© Yuriy Shamshin

1.1. SUMMARY OF CONVERSION RULES FOR RELATIONSHIP.

Cardinality	Membership Class	Number of Relations	Notes
1 : 1	Both Mandatory	**1**	all attributes in a single table
1 : 1	One Mandatory One Optional	**2**	Identifier of optional entity held in the mandatory entity relation
1 : 1	Both Optional	**3**	one relation for each entity and the relationship between them
1 : N	Both Mandatory	**2**	One relation for each entity and identifier of "one" end held in "many" end entity relation
1 : N	One Mandatory One Optional	**2**	*if the many end is mandatory*: one relation for each entity and identifier of the optional entity (the "one" end) held in the mandatory entity relation (the "many" end)
1 : N	One Mandatory One Optional	**3**	*if the many end is optional*: one relation for each entity and one for the relationship between them
1 : N	Both Optional	**3**	one relation for each entity and one for the relationship between them
M : N	Both Mandatory	**3**	one relation for each entity and one for the relationship between them
M : N	One Mandatory One Optional	**3**	one relation for each entity and one for the relationship between them
M : N	Both Optional	**3**	one relation for each entity and one for the relationship between them

←The following table provides a summary of the guidelines for converting components of an ERD into relations.

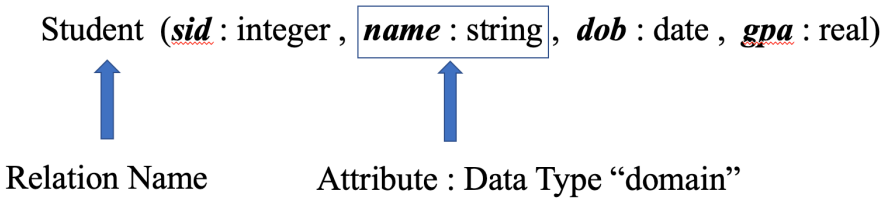
We need to be certain that if we store an identifier for one entity in a relation representing another entity, that the **identifier never has a null value**. If we have a null value for an identifier, we will never be able to find the other details that should be associated with it.

2. ER2RDM MAPPING RULES

2.0. THE DATABASE RELATION SCHEMA.

The schema specifies the relation's name, the name of each field (or column, or attribute), and the domain of each field.

The student information in a university database may be stored in a relation with the ***following schema***:



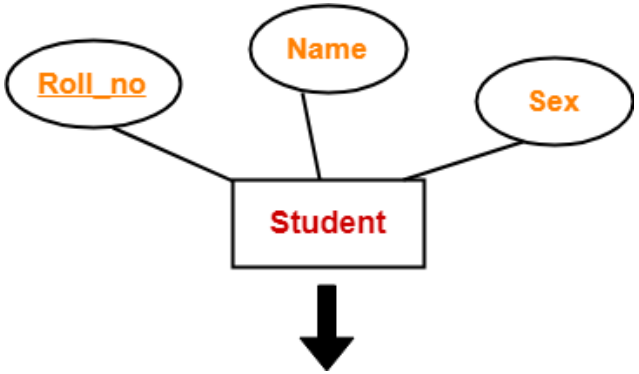
(GPA – Grade Point Average, DOB – Date of Birth)

Following 11 rules are used for converting an ER diagram into RM schema.

2.1. Rule-01: For Strong Entity Set With Only Simple Attributes

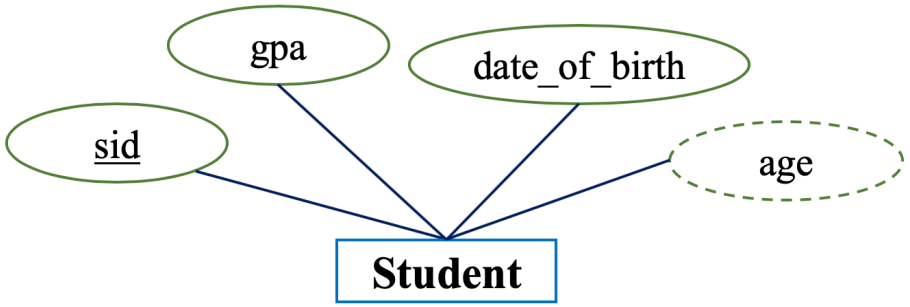
- A **strong entity set with only simple attributes** will require **only 1 table** in relational model.
- Attributes of the table will be the attributes of the entity set.
- Derived attributes removed from the table.
- The primary key of the table will be the key attribute of the entity set.

Examples



Roll_no	Name	Sex

Schema: Student(Roll_no, Name, Sex)



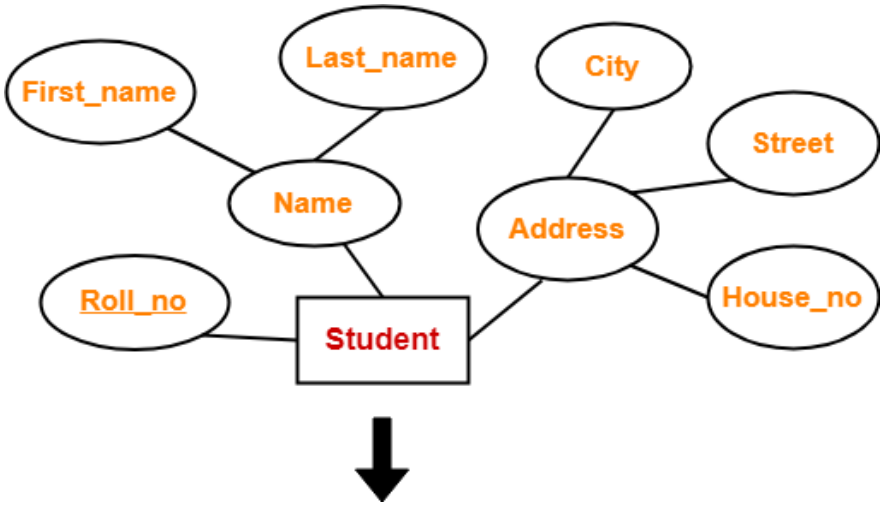
sid	gpa	date_of_birth

Schema: Student(sid, gpa, date_of_birth)

2.2. Rule-02: For Strong Entity Set With Composite Attributes

- A **strong entity set with any number of composite attributes** will require **only 1 table** in relational model.
- While conversion, simple attributes of the composite attributes are taken into account and not the composite attribute itself.

Example



Roll_no	First_name	Last_name	House_no	Street	City

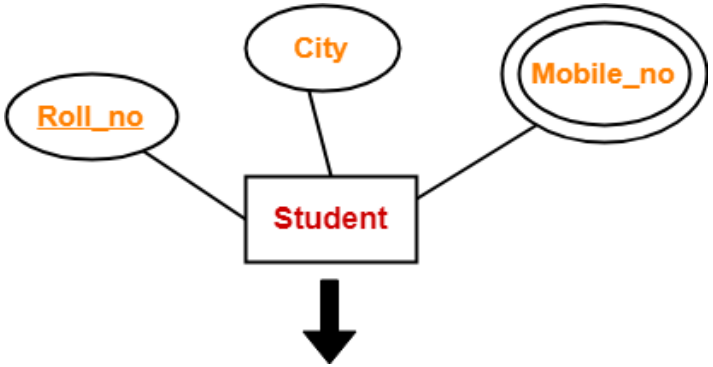
Schema: Student(Roll_no, First_name, Last_name, House_no, Street, City)

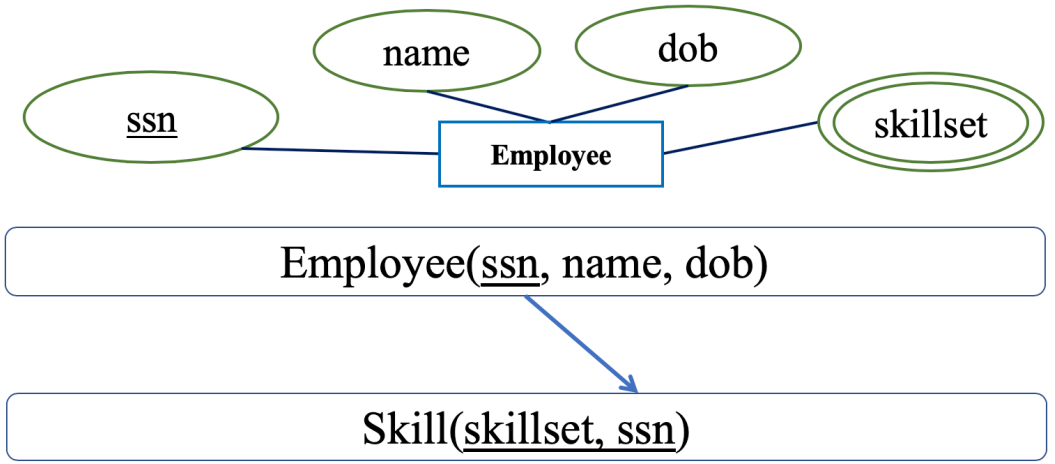
Roll numbers are unique identification numbers that can be assigned to students at the time of admission or after admission. To avoid instances of students of different batches (party) having the same roll numbers, you can set a common prefix to all student roll numbers of each batch (party).

2.3. Rule-03: For Strong Entity Set With Multivalued Attributes

- A **strong entity set with any number of multivalued attributes** will require **2 tables** in relational model.
- One table will contain all the simple attributes with the primary key.
- Other table will contain the primary key and all the multivalued attributes.

Examples



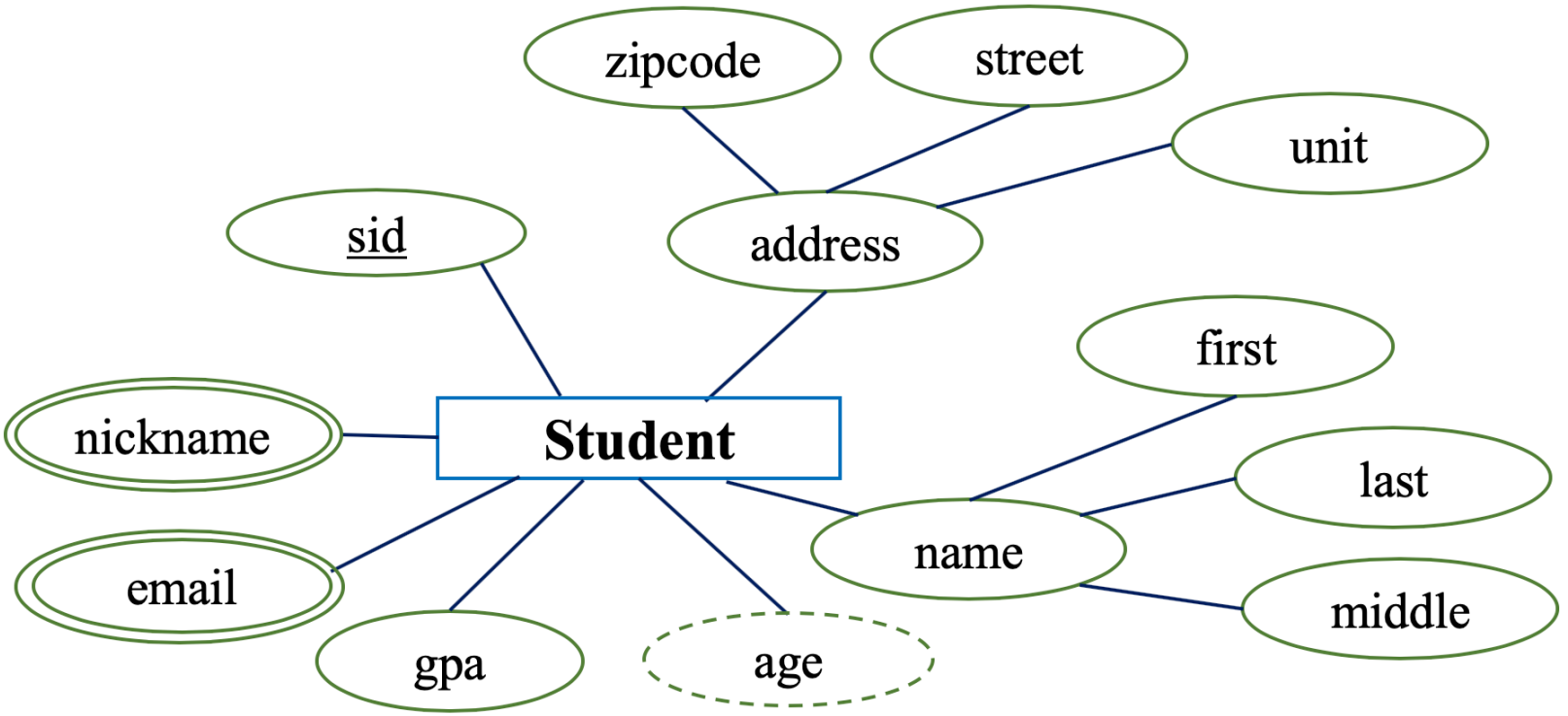


Key Attribute Migration (tranzition).

Roll_no	City
PK	

Roll_no	Mobile_no
PK,FK	PK

TASK. BUILD A RELATIONAL SCHEMA FROM THE FOLLOWING ER DIAGRAM:

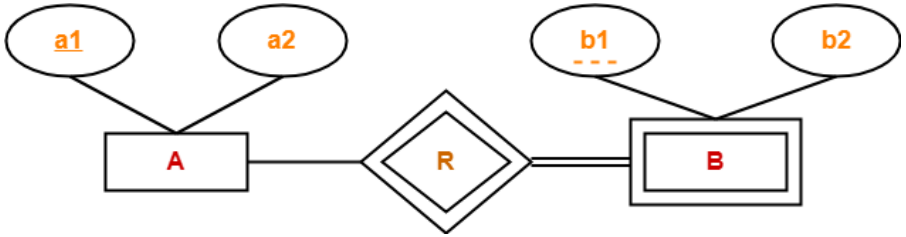


Student(<u>sid</u>, zipcodeAddr, streetAddr, unitAddr, firstName, middleName, lastName, gpa, dob)
Nickname(<u>sid</u>, <u>nickname</u>)
Email(<u>sid</u>, <u>email</u>)

2.4. Rule-04: For Binary Relationship With Weak Entity Set.

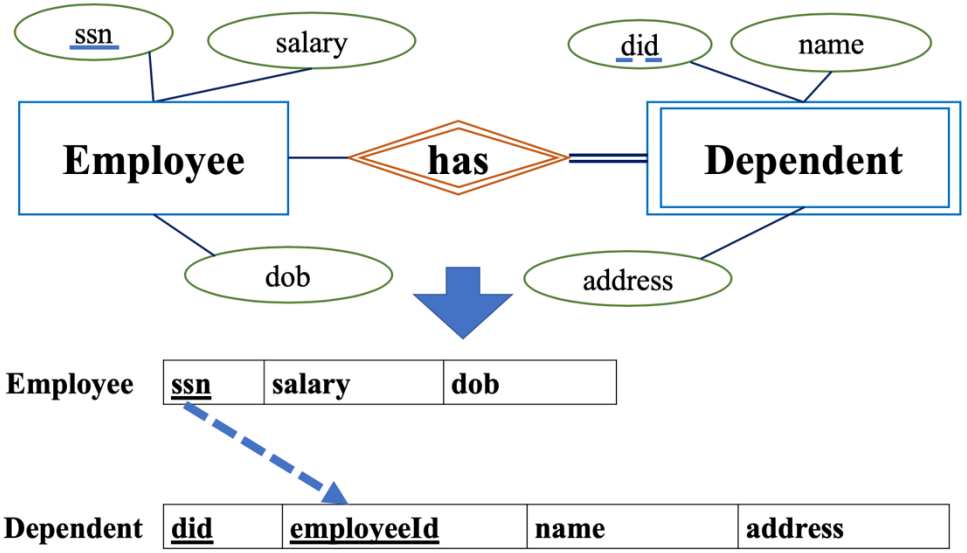
Weak entity set always appears in association with identifying relationship with total participation constraint.

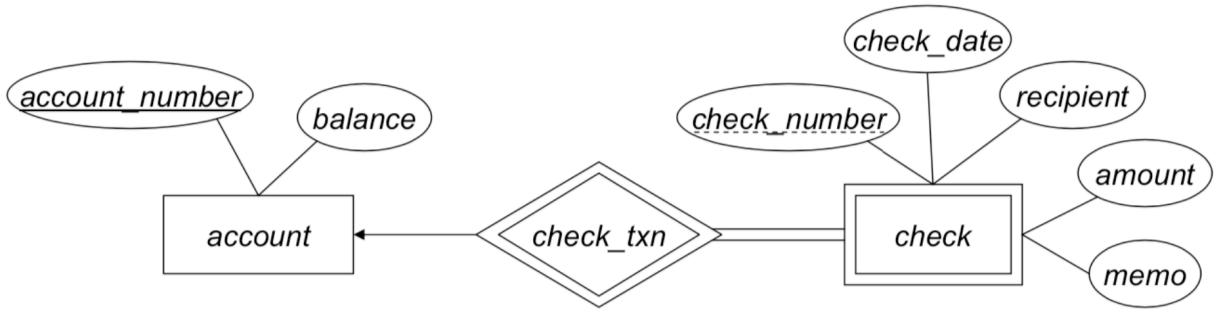
Examples



Here, **2 tables** will be required

1. A(a1, a2)
2. BR(a1, b1, b2)





account(*account_number*, *balance*)

check schema:

Discriminator is *check_number*

Primary key for *check* is: (*account_number*, *check_number*)

check(*account_number*, *check_number*, *check_date*, *recipient*, *amount*, *memo*)

2.5. Rule-05: Translating Relationship Set Without Cardinality Ratios Description into a Table

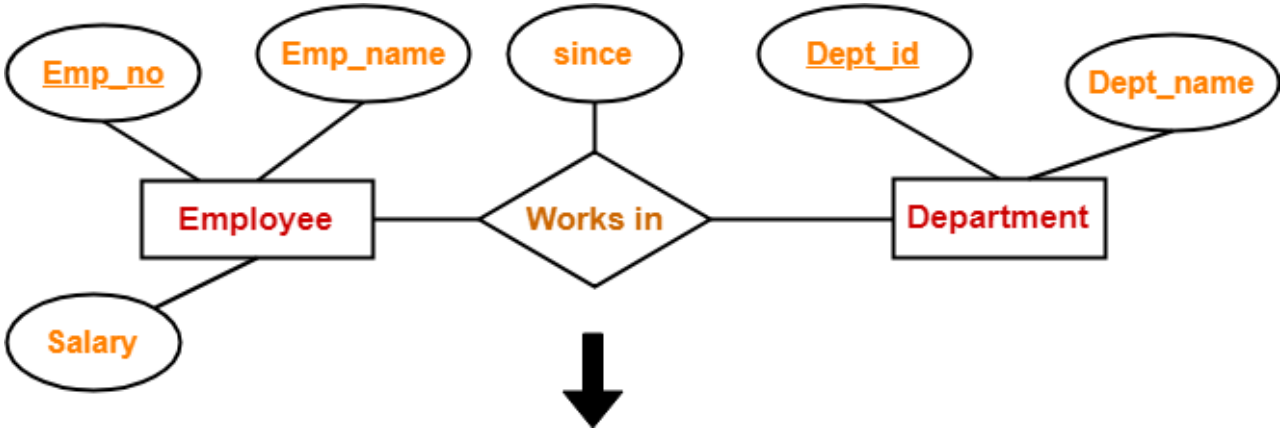
A relationship between 2 relations will require **3 tables** in the relational model.

Attributes of the table are

- Primary key attributes of the participating entity sets
- Its own descriptive attributes if any.

Set of non-descriptive attributes will be the primary key.

Example



Emp_no	Dept_id	since

Schema: WorksIn(Emp_no, Dept_id, since)

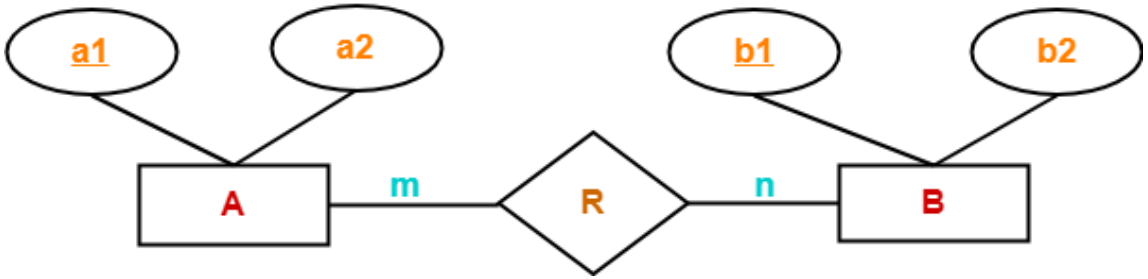
Note.

If we consider the overall ER diagram, 3 tables will be required

- One table for the entity set “Employee”
- One table for the entity set “Department”
- One table for the relationship set “WorksIn”

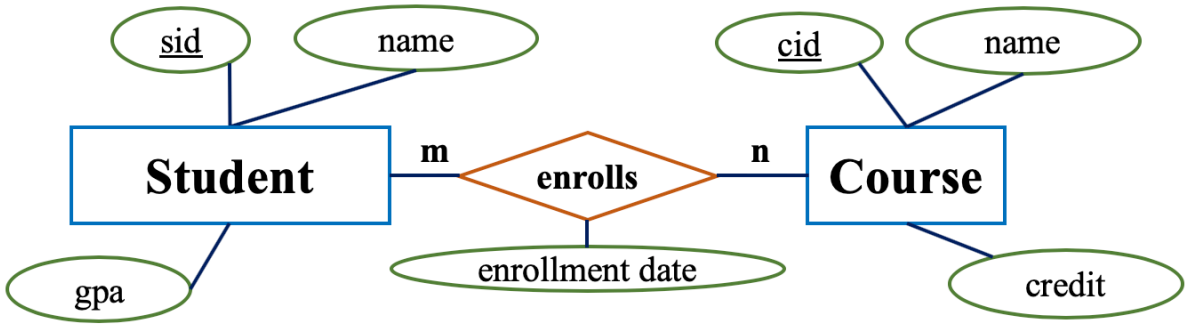
2.6. Rule-06: For Binary Relationship With Cardinality Ratio M:N

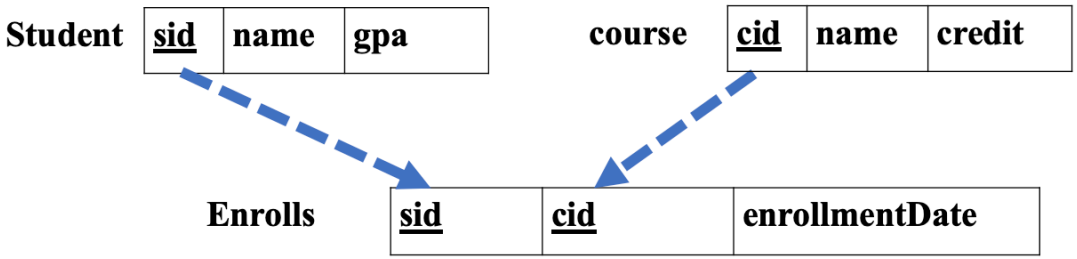
Examples



Here, **3 tables** will be required

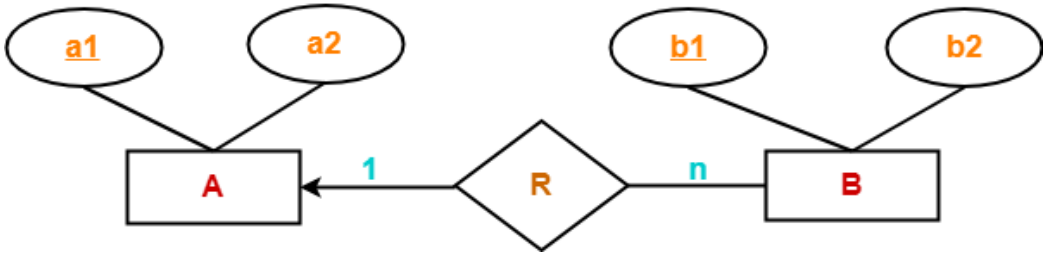
1. A(a1, a2)
2. R(a1, b1)
3. B(b1, b2)

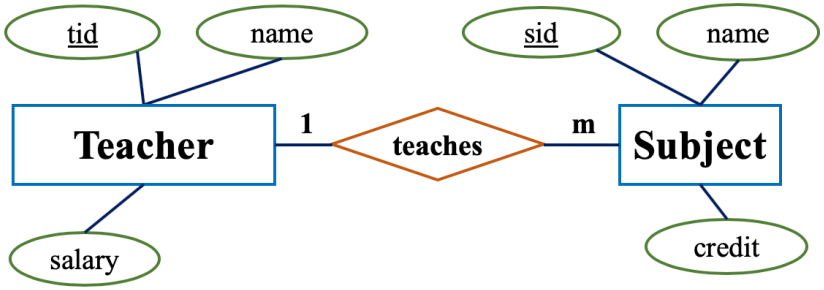


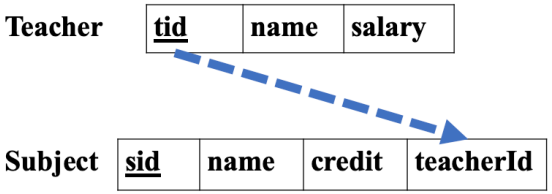


2.7. Rule-07: For Binary Relationships With Cardinality Ratio 1:1,N or 1,M:1

2.7.1. Case-01: For Binary Relationship With Cardinality Ratio 1:1,N





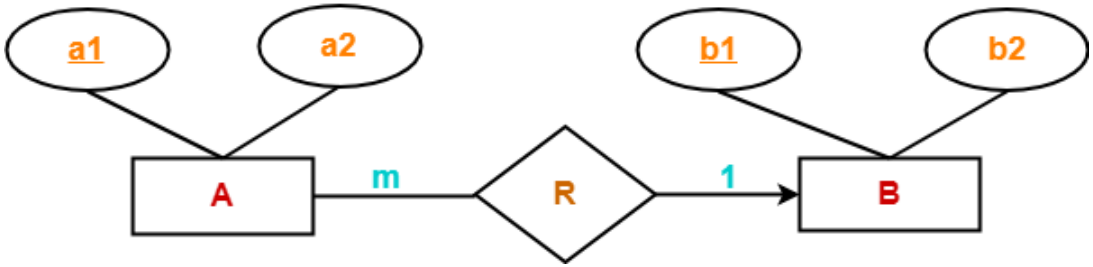


Here, 2 tables will be required

1. A(<u>a1</u>, a2)
2. BR(<u>b1</u>, b2, a1)

<u>NOTE.</u> Here, combined table will be drawn for the entity set B and relationship set R.

2.7.2. Case-02: For Binary Relationship With Cardinality Ratio 1,M:1

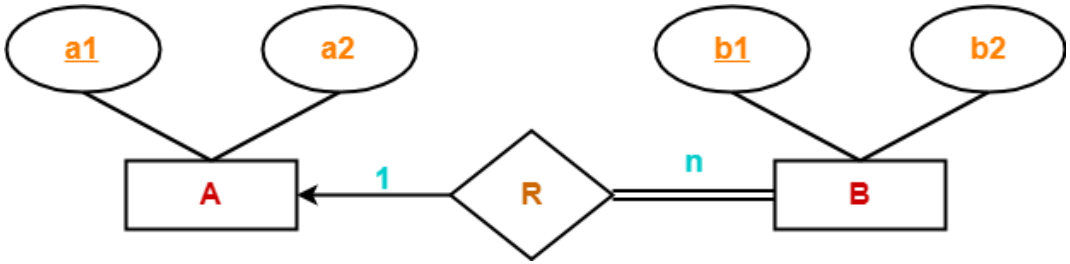


Here, 2 tables will be required

1. AR(<u>a1</u>, a2, b1)
2. B(<u>b1</u>, b2)

<u>NOTE.</u> Here, combined table will be drawn for the entity set A and relationship set R.

2.8. Rule-08: For Binary Relationship With Cardinality Ratio 1:0,n

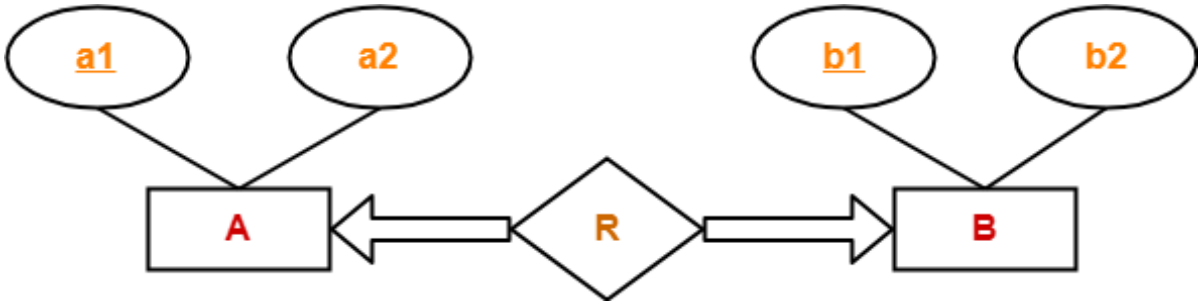


Then, **3 tables** will be required

1. A(a1, a2)
2. R(a1, b1)
3. B(b1, b2)

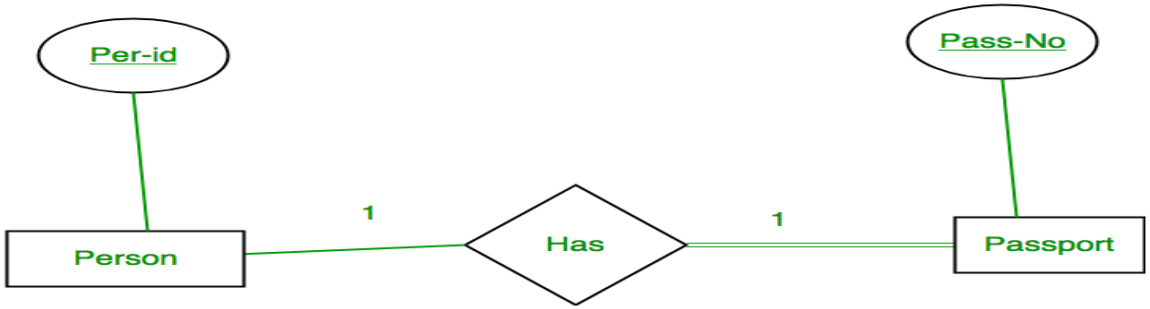
2.9. Rule-09: For Binary Relationship With Cardinality 1,1;1,1.

If there is a key constraint from both the sides of an entity set with total participation, then that binary relationship is represented using only single table.

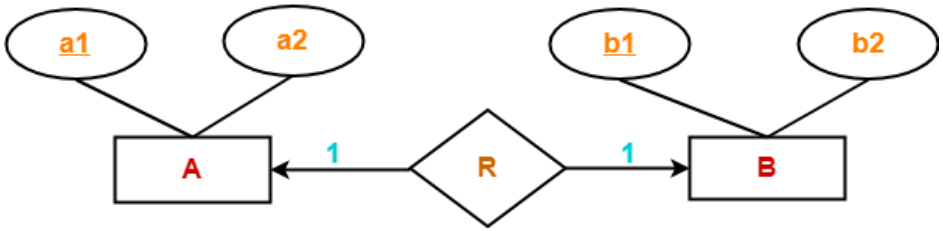


Here, Only one table is required

- ARB(a1, a2, b1, b2)



2.10. Rule-10: For Binary Relationships With Cardinality Ratio 1,1:0,1 or 0,1:1,1



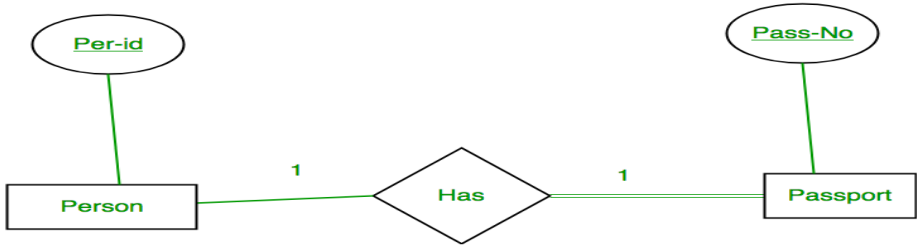
Here, **2 tables** will be required. Either combine 'R' with 'A' or 'B'

Way-01: for 1,1;0,1

1. AR(a1, a2, b1)
2. B(b1, b2)

Way-02: for 0,1;1,1

1. A(a1, a2)
2. BR(a1, b1, b2)

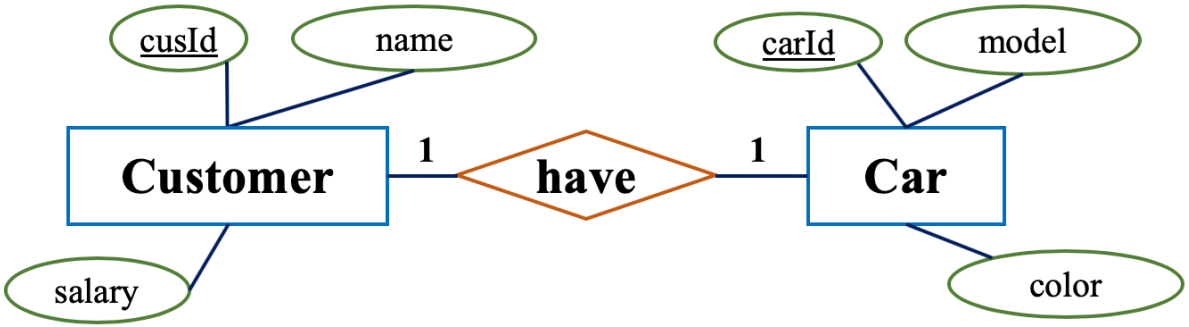


A person has 0 or 1 passport number and Passport is always owned by 1 person. So it is 1:1 cardinality with full participation constraint from Passport (1,1:0,1).

2.11. Rule-11: For Binary Relationships With Cardinality Ratio 0,1:0,1

Then, **3 tables** will be required

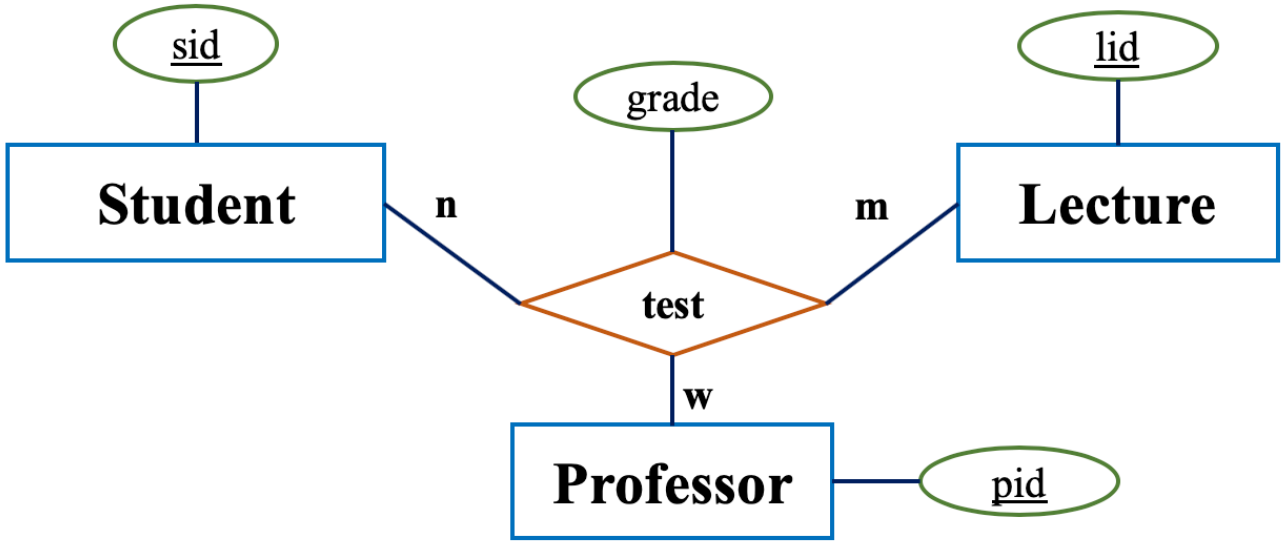
1. A(a1, a2)
2. R(a1, b1)
3. B(b1, b2)



Then, **3 tables** will be required

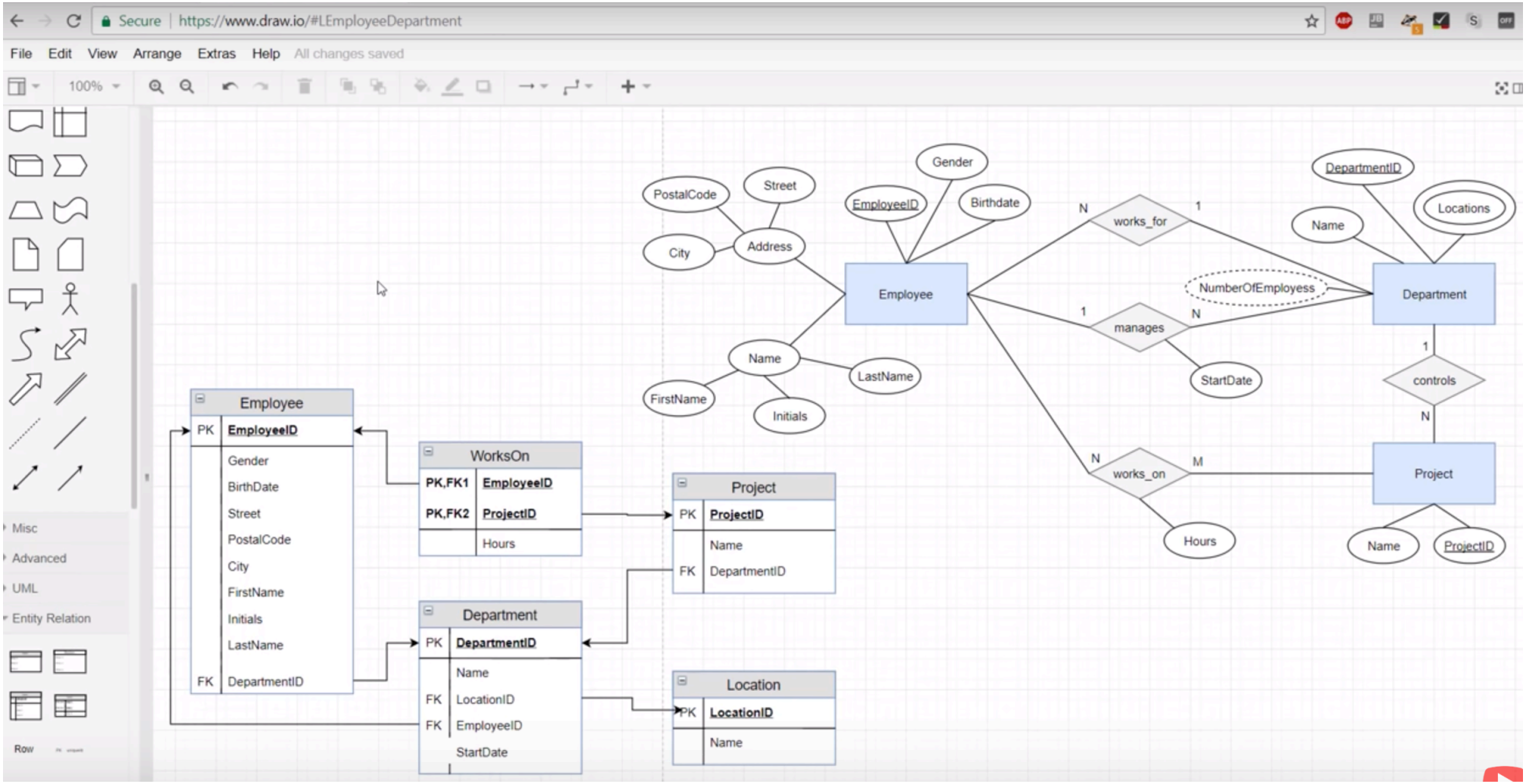
1. Customer(cusId, name, salary)
2. CusCar(cusId, carId)
3. Car(carId, model, color)

2.12. Ternary Relationship Example n:m:w



Test	sid	lid	pid	grade

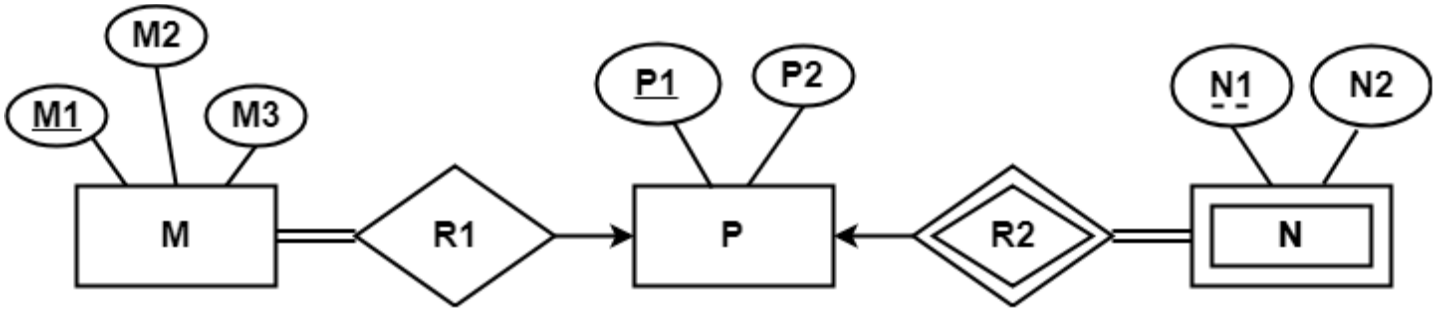
2.13. Example of convert an ER diagram to the Relational Data Model.



3. Practice problem based on Converting ER Diagram to Tables

3.1. Problem-01:

Find the minimum number of tables required for the following ER diagram in relational model



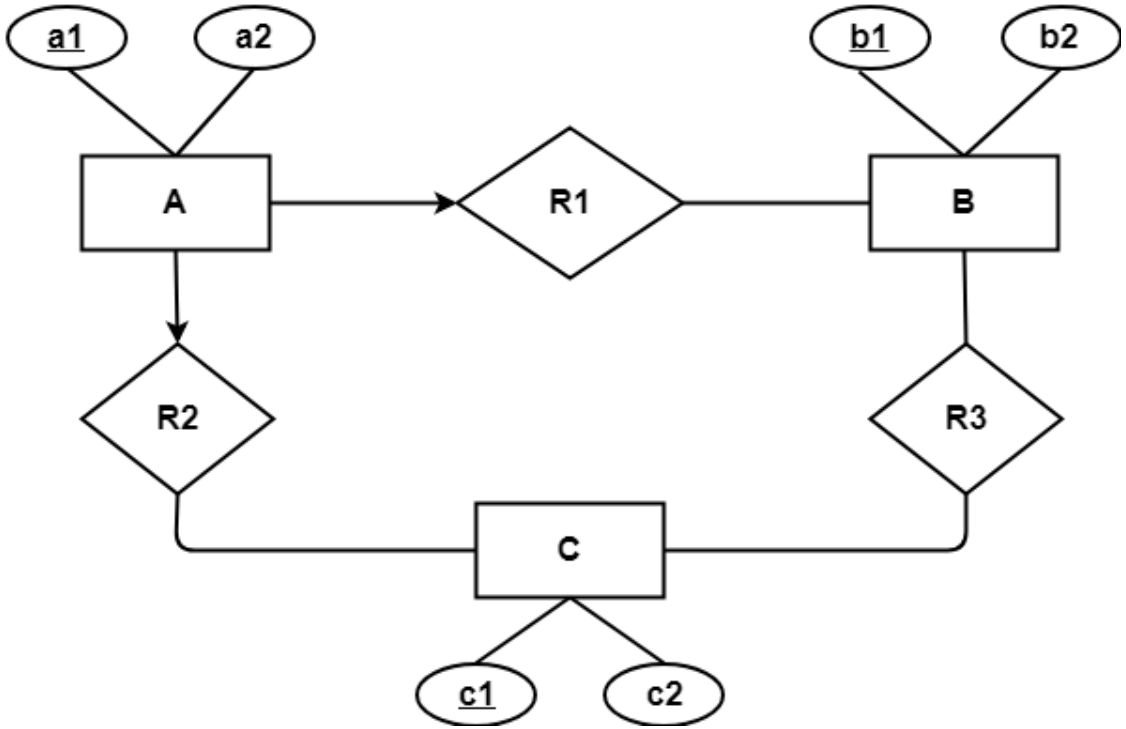
3.1.1. Solution

Applying the rules, minimum 3 tables will be required

- MR1(M1, M2, M3, P1)
- P(P1, P2)
- NR2(P1, N1, N2)

3.2. PROBLEM-02:

Find the minimum number of tables required to represent the given ER diagram in relational model



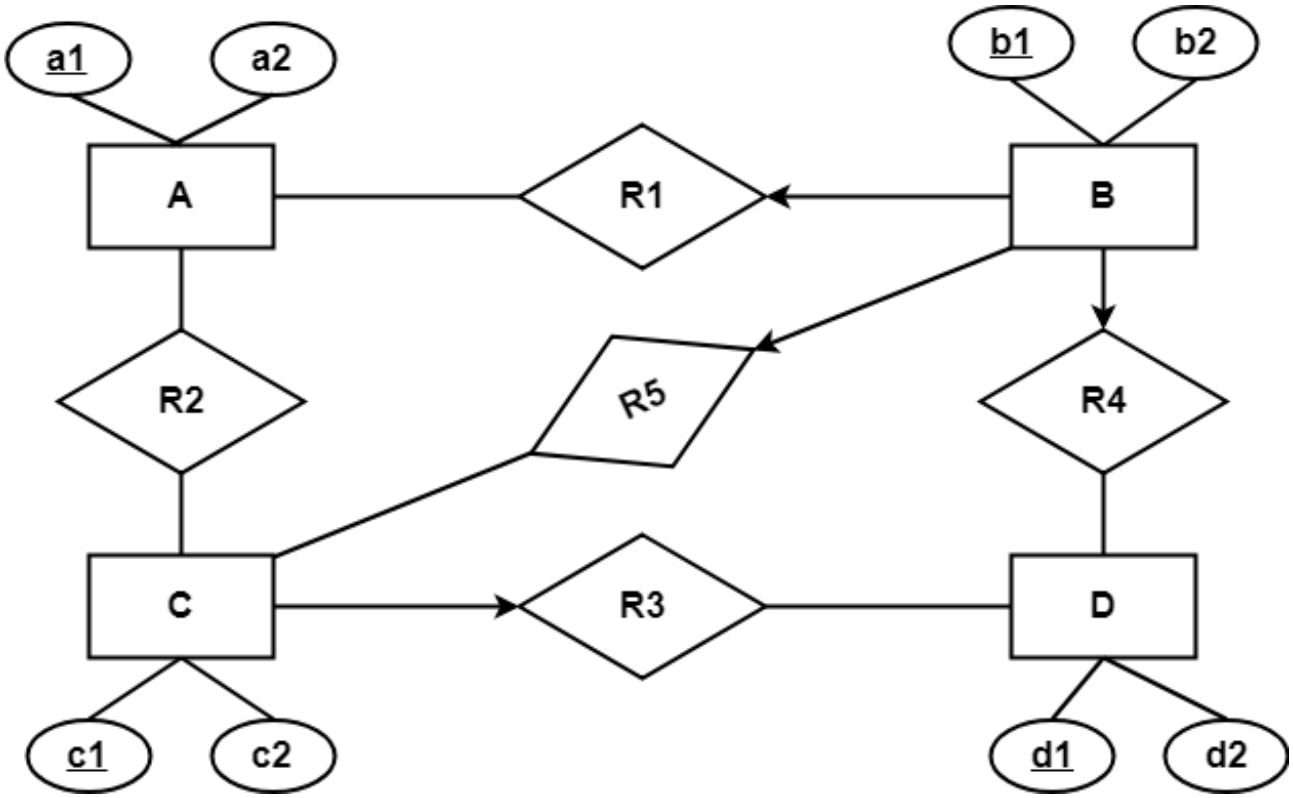
3.2.1. SOLUTION

Applying the rules, minimum 4 tables will be required

- AR1R2(a1, a2, b1, c1)
- B(b1, b2)
- C(c1, c2)
- R3(b1, c1)

3.3. PROBLEM-03:

Find the minimum number of tables required to represent the given ER diagram in relational model



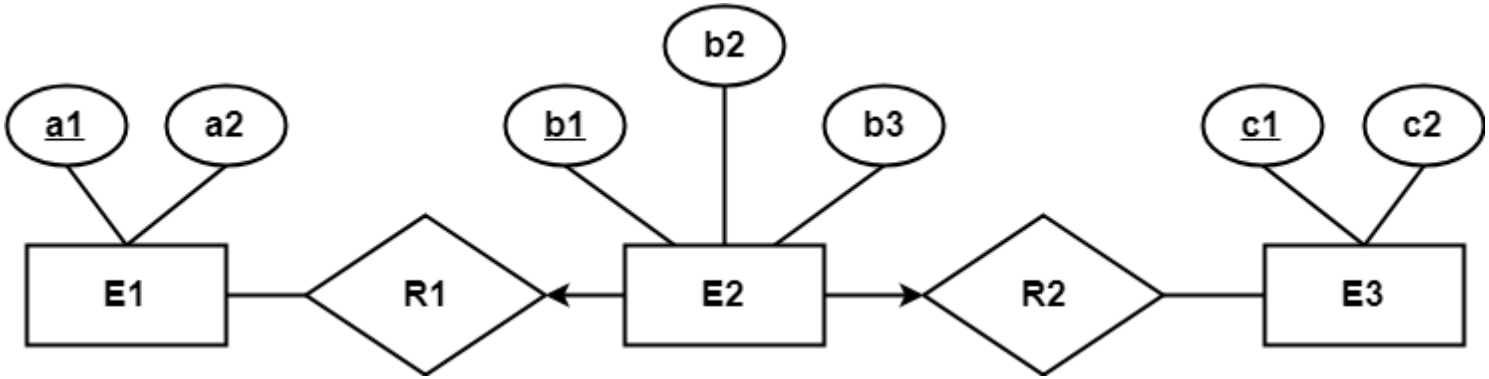
3.3.1. SOLUTION

Applying the rules, minimum 5 tables will be required

- BR1R4R5(b1, b2, a1, c1, d1)
- A(a1, a2)
- R2(a1, c1)
- CR3(c1, c2, d1)
- D(d1, d2)

3.4. PROBLEM-04:

Find the minimum number of tables required to represent the given ER diagram in relational model



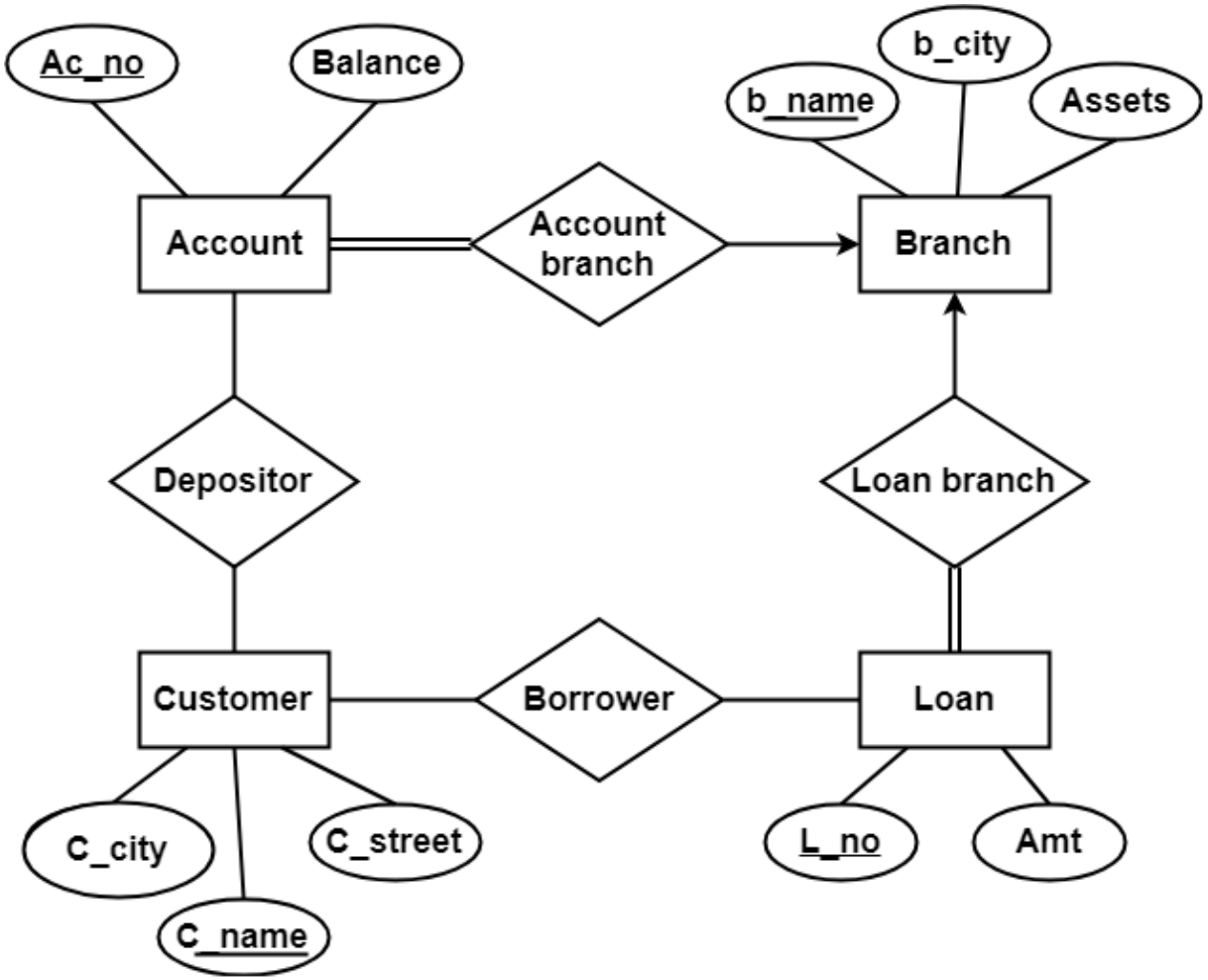
3.4.1. SOLUTION

Applying the rules, minimum 3 tables will be required

- E1(a1, a2)
- E2R1R2(b1, b2, a1, c1, b3)
- E3(c1, c2)

3.5. PROBLEM-05:

Find the minimum number of tables required to represent the given ER diagram in relational model



3.5.1. SOLUTION

Applying the rules that we have learnt, minimum 6 tables will be required

- Account (Ac_no, Balance, b_name)
- Branch (b_name, b_city, Assets)
- Loan(L_no, Amt, b_name)
- Borrower(C_name, L_no)
- Customer(C_name, C_street, C_city)
- Depositor(C_name, Ac_no)

© Yuriy Shamshin